

Capas de colaboraciones. Una colaboración se puede expresar en diferentes niveles de granularidad. Una colaboración de grano grueso puede refinarse para producir otra colaboración de grano más fino, expandiendo una o más operaciones de una colaboración de alto nivel en diferentes colaboraciones de más bajo nivel, una por cada operación.

Una colaboración se puede implementar utilizando colaboraciones subordinadas, de forma que cada colaboración subordinada implemente una parte de la funcionalidad global y tenga su propio conjunto de roles. Cada rol de la colaboración global puede ligarse a uno o más roles de las composiciones anidadas. Si un rol de nivel externo se liga a más de un rol de nivel interno, entonces implícitamente conecta el comportamiento de las colaboraciones de más bajo nivel. Ésta es la única forma de interacción entre casos de uso. Un enfoque de diseño significa trabajar de dentro hacia fuera: primero construir los roles más internos y reducidos, y luego irlos combinando para producir roles más amplios y externos que tengan múltiples responsabilidades.

Ligadura en tiempo de ejecución. En tiempo de ejecución, los objetos y enlaces están ligados a los roles de la colaboración. Un objeto puede estar ligado a uno o más roles, normalmente en diferentes colaboraciones. Si un objeto está ligado a múltiples roles, entonces se dice que representa una interacción “accidental” entre los roles —esto es, una interacción que no es inherente a los propios roles, sino un efecto lateral de su uso en un contexto más amplio—. A menudo, un objeto desempeña roles en más de una colaboración como parte de una colaboración más amplia. Este solapamiento entre colaboraciones proporciona un flujo implícito de control y de información entre ellas.

Estructura

Roles. Una colaboración contiene un conjunto de roles, cada uno de los cuales es una referencia a uno o más clasificadores base (rol de clasificador) o asociaciones base (rol de asociación). Un rol es una ranura en la colaboración que describe un uso de un clasificador o una asociación dentro de la colaboración. Un rol es también un clasificador, pues un objeto ligado al rol en una instancia de la colaboración es una instancia transitoria del rol. Dentro de una instancia de la colaboración, se liga un objeto a cada rol de clasificador y un enlace a cada rol de asociación. Se puede ligar un objeto a más de un rol de clasificador de una misma instancia de una colaboración, aunque no es lo normal, e incluso puede ser deseable evitarlo mediante una restricción. Los objetos de la misma clase pueden aparecer en múltiples roles de la misma instancia de colaboración. Cada clasificador puede mantener una lista de las características de clasificador utilizadas en la colaboración. Las otras características son irrelevantes en la colaboración, si bien pueden ser utilizadas en otras colaboraciones. Si un mismo clasificador base está implicado en múltiples roles, éstos deberían tener un nombre que permitiera distinguirlos, mientras que si sólo hay un uso del clasificador en una colaboración, el rol puede ser anónimo ya que basta el clasificador para identificarlo. Los roles definen la estructura de la colaboración.

Si hay clasificación múltiple, el rol puede tener múltiples clasificadores base, de forma que un objeto ligado al rol de clasificador será una instancia de cada uno de ellos.

Generalizaciones. Una colaboración también puede incluir generalizaciones y restricciones, que se añaden a las relaciones que puedan tener los clasificadores participantes fuera de la colaboración. Una generalización es necesaria en una colaboración sólo cuando los clasificadores de la colaboración sean parámetros; si no, su estructura de generalización se especificará como parte de su definición como clasificadores y no podrá ser alterada por la colaboración. En una colaboración parametrizada (un patrón) algunos de los roles de clasificador pueden ser

parámetros. Una generalización entre dos roles de clasificador parametrizados indica que todos los clasificadores ligados a los roles deben satisfacer la relación de generalización (el clasificador ligado al rol padre debe ser un antecesor del clasificador ligado al rol hijo, pero no es necesario que sean padre e hijo).

Por ejemplo, el patrón **Composite** (**Compuesto**) de [Gamma-95] representa un árbol de objetos recursivo en el que la clase **Componente** es un elemento genérico del árbol, **Compuesto** es un elemento recursivo y **Hoja** es un elemento hoja (Figura 13.45). **Componente** es el padre de **Hoja** y **Compuesto**, el último de los cuales es un agregado de elementos **Componente** (he aquí la recursividad). **Componente**, **Compuesto** y **Hoja** son parámetros del patrón que son sustituidos por clases reales cuando se utiliza el mismo. Cualquier conjunto de clases reales ligado al patrón debe observar la relación antecesor-sucesor entre **Componente** y sus hijos **Compuesto** y **Hoja**. Algunos ejemplos de sustitución podrían ser **Gráfico**, **Dibujo** y **Rectángulo**; **EntradaDeDirectorio**, **Directorio** y **Archivo**; y otras clases recursivas. Si una ligadura no cumple la restricción de generalización, estará mal formada.

Restricciones. Las restricciones se pueden definir sobre roles parametrizados y no parametrizados. Estas restricciones se añadirán a las que puedan existir sobre los clasificadores ligados a los roles. Las restricciones se aplican a todas las instancias de la colaboración.

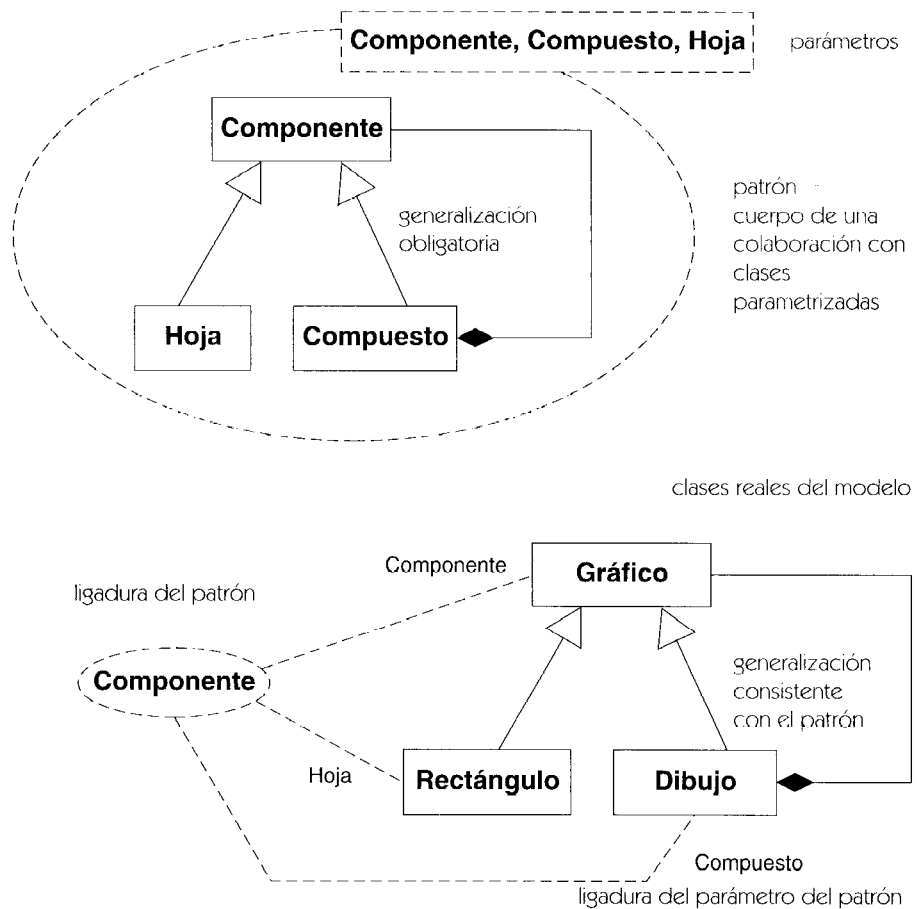


Figura 13.45 Patrón: una colaboración parametrizada

Mensajes. Una colaboración puede tener un conjunto de mensajes para describir su comportamiento dinámico. Una colaboración con mensajes es una interacción. Puede haber múltiples interacciones, cada una de las cuales describe parte de una misma colaboración. Una interacción puede describir la implementación de una operación mientras otra describe otra operación, por ejemplo. Cada mensaje contiene información que indica el orden del mismo en la secuencia de mensajes; dicha información equivale a la especificación de una máquina de estados disparada por mensajes.

Origen. Una colaboración puede representar una clase, caso de uso o método (una colaboración es la implementación de una operación, no su especificación). La colaboración describe el comportamiento del objeto origen.

Notación

Un diagrama de colaboración es un grafo de símbolos de clase (rectángulos) que representan roles de clasificador y rutas de asociación (líneas continuas) que representan roles de asociación, con símbolos de mensaje asociados a sus caminos de roles de asociación. Un diagrama de colaboración sin mensajes muestra el *contexto* en el que pueden ocurrir interacciones, sin mostrar ninguna interacción. Puede utilizarse para mostrar el contexto de un evento u operación simple para todas las operaciones de una clase o grupo de clases. Si existen mensajes asociados a las líneas de asociación, el diagrama muestra una interacción. Típicamente, una interacción representa la implementación de una operación o caso de uso.

Un diagrama de colaboración muestra ranuras para los objetos implicados en la misma, mediante roles de clasificador. Un rol de clasificador se distingue de un clasificador porque tiene un nombre y una clase, con la sintaxis nombreRol : nombreClase. Se puede omitir el nombre de la clase o el del clasificador, pero los dos puntos son necesarios. Un diagrama también muestra los enlaces entre los objetos como roles de asociación, incluyendo los enlaces transitorios que representan argumentos de procedimientos, variables locales y *auto*-enlaces. Varios roles de asociación pueden tener el mismo nombre de asociación, suponiendo que conectan diferentes roles de clasificador. Las flechas situadas en las líneas de los enlaces indican la dirección de navegación, de manera que una punta de flecha sobre una línea que une cajas de objetos indica un flujo de mensajes que fluye en la dirección dada, no pudiendo fluir los mensajes en sentido contrario al especificado en un enlace de un solo sentido, por lo que los flujos de mensajes deben ser compatibles con las flechas de navegación.

Generalmente, no se muestran de forma explícita los valores de atributos individuales en los roles de clasificador. Si hay que enviar mensajes a valores de los atributos, dichos atributos deberían modelarse como objetos que utilizan asociaciones.

Se pueden emplear herramientas u otras aplicaciones gráficas para sustituir o complementar las palabras clave. Por ejemplo, cada tipo de vida podría mostrarse con un color diferente. Es más, la herramienta podría incluso utilizar la animación para mostrar la creación y destrucción de elementos y el estado del sistema en diferentes momentos.

Implementación de una operación

Una colaboración que muestra la implementación de una operación incluye símbolos para el rol de objeto origen y para los roles de otros objetos que éste utiliza, directa o indirectamente, para

llevar a cabo la operación. Los mensajes que aparecen sobre los roles de asociación representan el flujo de objeto de una interacción.

Una colaboración que describe una operación también incluye símbolos de rol que representan los argumentos de la operación, y variables locales creadas durante su ejecución. Los objetos creados durante la ejecución pueden designarse mediante **{new}**, los objetos destruidos durante la misma se pueden designar mediante **{destroyed}**, y los creados durante la ejecución y destruidos durante su transcurso se pueden designar utilizando **{transient}**. Los objetos sin palabra clave existen ya al iniciarse la operación y siguen existiendo tras su terminación.

Los mensajes internos que implementan un método están numerados comenzando a partir del número 1. En un flujo de objeto procedimental, los números asociados a los mensajes emplean una secuencia de números anidada utilizando la notación de puntos en concordancia con los niveles de anidamiento de llamadas. Por ejemplo, el segundo paso del nivel superior llevará asociado el número 2; el primer paso subordinado dentro de éste será el mensaje 2.1. En los mensajes asíncronos intercambiados entre objetos concurrentes, todos los números de secuencia están al mismo nivel (es decir, no están anidados).

Véase mensaje, donde se incluye una completa descripción de la sintaxis de los mensajes que incluye los números de secuencia.

Un diagrama de colaboración completo muestra los roles de todos los objetos y enlaces utilizados por la operación. Si no se muestra un objeto, se asume que no se utiliza. Sin embargo, no es seguro asumir que una operación utiliza todos los objetos que aparecen en un diagrama de colaboración.

Ejemplo

En la Figura 13.46 se invoca la operación **actualizarVisualización** sobre un objeto **Controlador**. En el momento de invocarse la operación, ésta ya tiene un enlace al objeto **Ventana** donde se va a mostrar la imagen. También tiene un enlace a un objeto **Cable**, el objeto cuya imagen será visualizada en la ventana.

La implementación de más alto nivel de la operación **actualizarVisualización** tiene un solo paso —la invocación a la operación **mostrarPosiciones** del objeto **cable**—. Esta operación tiene el número de secuencia 1 puesto que se trata del primer método de nivel superior. Este mensaje fluye a través de una referencia al objeto **Ventana** que se necesitará más adelante.

La operación **mostrarPosiciones** llama a la operación **dibujarSegmento** del propio objeto **cable**. La llamada, etiquetada con el número de secuencia 1.1, se trata a través del enlace implícito **self**. El asterisco indica una llamada iterativa a la operación cuyos detalles se muestran entre corchetes.

Cada operación **dibujarSegmento** accede a dos objetos **SegmentoCable**, uno indexado por el valor **i-1** y otro por el valor **i**. Aunque sólo hay una asociación entre **Cable** y **SegmentoCable**, en el contexto de esta operación son necesarios dos enlaces a dos objetos **Cable**. Los objetos están etiquetados como **izquierdo** y **derecho**, que son los roles de clasificador en la colaboración. Cada mensaje fluye por cada uno de los enlaces, etiquetándose como mensaje 1.1.1a y 1.1.1b, lo que indica que son pasos de la operación 1.1. Las letras del final indican que los dos mensajes pueden ser tratados concurrentemente. En una implementación normal, probablemente no

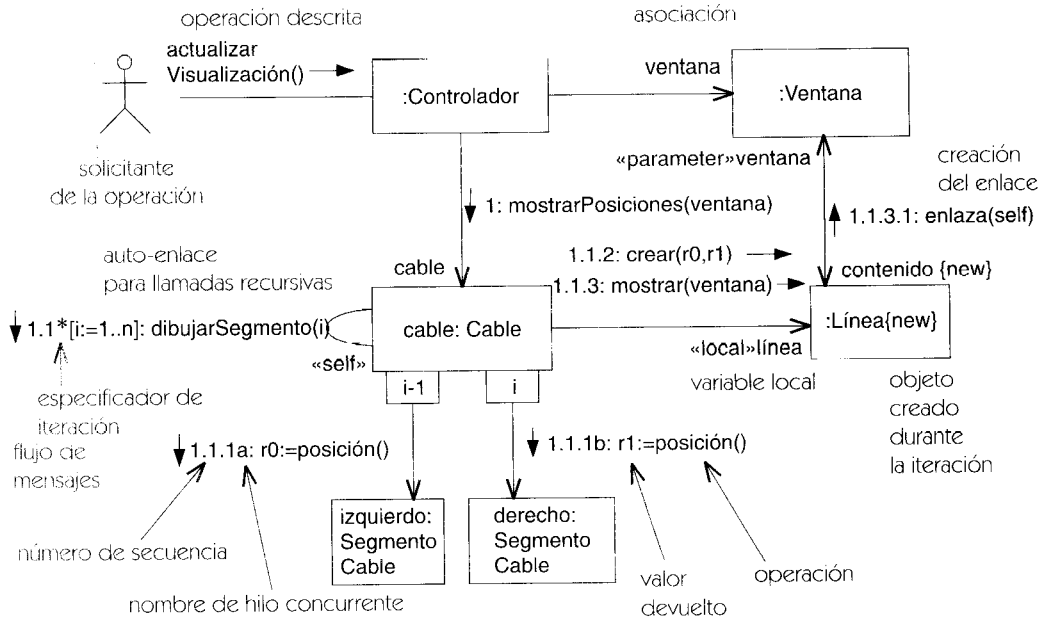


Figura 13.46 Diagrama de colaboración con flujo de mensajes

se ejecutarían en paralelo, pero como se han declarado como concurrentes pueden ejecutarse en cualquier orden de secuencia.

Cuando ambos valores (**r0** y **r1**) han sido devueltos, puede continuar el siguiente paso de la operación 1.1. El mensaje 1.1.2 es un mensaje de creación que se envía a un objeto **Línea**. Realmente se envía a la propia clase **Línea** (al menos en principio), que crea un nuevo objeto **Línea** enlazado al objeto que envió el mensaje. El nuevo objeto tiene la etiqueta **{new}** para indicar que ha sido creado durante la operación pero que sigue viviendo después. El nuevo enlace tiene la etiqueta **«local»** para indicar que se trata de una variable local al procedimiento. Las variables locales son inherentemente transitorias, por lo que desaparecen cuando finaliza el procedimiento y por tanto no es necesario etiquetarlas con la palabra clave **{transient}**.

El paso 1.3 utiliza el enlace recién creado para enviar un mensaje mostrar al recién creado objeto **Línea**. El puntero al objeto **ventana** se pasa como argumento, de forma que el objeto **Línea** pasa a tener acceso al mismo a través de un enlace **«parameter»**. Obsérvese que el objeto **Línea** tiene un enlace al mismo objeto **ventana** que está asociado con el objeto original **Controlador**; esto es importante para la operación y por ello se muestra en el diagrama. En el paso final, 1.3.1, se solicita al objeto **Ventana** la creación de un enlace al objeto **Línea**. Este enlace es una asociación, por lo que lleva el nombre de rol **contenido**, y está etiquetado como **{new}**.

El estado final del sistema puede observarse eliminando mentalmente todos los enlaces temporales. Existe un enlace desde **Controlador** hasta **cable** y desde **cable** hasta sus **SegmentoCable**, un enlace de **Controlador** a **ventana** y otro desde **ventana** hasta su **contenido**. Una vez terminada la operación, una **Línea** no tiene acceso a la **Ventana** que la contiene, por lo que el enlace en esa dirección es transitorio y desaparece al finalizar la operación. De forma similar, un objeto **Cable** no tiene por qué seguir teniendo acceso a los objetos **Línea** utilizados para mostrarlo.

colaboración entre instancias

Las colaboraciones se pueden expresar como descriptores y como instancias, al igual que muchos otros elementos del modelo. Una colaboración entre descriptores muestra una relación potencial entre objetos, pudiendo instanciarse muchas veces la colaboración para producir instancias de colaboración. Cada instancia de colaboración muestra una relación entre objetos específicos: es una colaboración entre instancias.

Pero, ¿qué forma debería utilizarse para modelar una determinada situación? Si el diagrama muestra contingencia, debe emplearse un diagrama de descriptores, ya que los diagramas de objetos no tienen contingencia, es decir, no tienen estructuras condicionales ni bucles, ya que estos elementos forman parte de las descripciones genéricas. Una instancia no tiene un rango de valores ni un posible conjunto de caminos de control: tiene un valor concreto y una historia de control.

Si el diagrama muestra valores específicos de atributos o argumentos, si muestra un número específico de objetos o enlaces de entre los muchos posibles en una multiplicidad de tamaño variable, o si muestra una elección particular de bifurcaciones y bucles durante una ejecución, debe ser un diagrama de instancias.

En muchos casos es posible utilizar las dos formas, generalmente cuando la ejecución no tiene bucles. En estos casos cualquier ejecución es prototípica, y no hay mucha diferencia entre la forma de descriptor y la forma de instancia.

combinación

Tipo de relación que relaciona dos partes de la descripción de un clasificador, combinándolas para producir el descriptor completo del elemento.

Véase extender, incluir.

Semántica

Una de las potentes capacidades de la orientación a objetos es la posibilidad de combinar descripciones de elementos del modelo a partir de piezas incrementales. La herencia combina clasificadores relacionados por la generalización para producir el descriptor efectivo completo de una clase.

Otras formas de combinación de descriptores son las relaciones de extensión e inclusión. Estas relaciones se modelan como variaciones de la relación de combinación. La generalización también podría englobarse en esta misma categoría, aunque debido a su importancia se trata como una relación fundamental diferente.

Notación

Una relación de combinación se representa mediante una flecha con línea discontinua con una palabra clave de estereotipo asociada. *Véase* extender e incluir para ver los detalles específicos de cada una.

Discusión

Son posibles otros tipos de relación de combinación. Algunos lenguajes de programación, como CLOS, implementan algunas potentes variantes de combinación de métodos.

comentario

Anotación asociada a un elemento o colección de elementos. Un comentario no tiene significado directo, pero puede mostrar información semántica u otra información significativa para el que realiza el modelado o para la herramienta, por ejemplo una restricción o el cuerpo de un método.

Véase también nota.

Semántica

Un comentario está formado por una cadena de texto pero también puede incluir documentos embebidos si lo permite la herramienta de modelado utilizada. Un comentario puede asociarse con un elemento del modelo, con un elemento de presentación o con un conjunto de elementos. Proporciona una descripción textual de información arbitraria, pero no tiene impacto semántico. Los comentarios proporcionan información a quienes realizan el modelo y pueden utilizarse para buscar modelos.

Notación

Los comentarios se muestran en símbolos de nota, que se representan mediante rectángulos con la esquina superior derecha doblada en forma de “oreja de perro” asociados mediante una línea o líneas discontinuas al elemento o elementos al que se aplica el comentario (Figura 13.47). Las herramientas de modelado pueden proporcionar libremente formas adicionales de mostrar comentarios y navegar por ellos, como comentarios *pop-up*, tipos de letra especiales, etc.

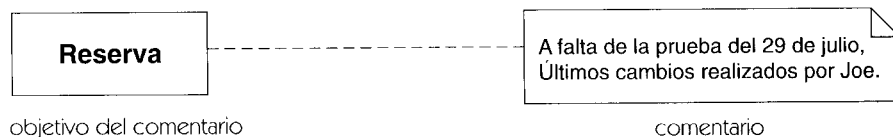


Figura 13.47 Comentario

Elementos estándar

requirement, responsibility.

comillas

Los corchetes angulares pequeños (« ») se emplean para denotar cita en Francés, Italiano, Español y otros idiomas. En la notación UML se emplean para denotar palabras reservadas y nombres de estereotipos. Por ejemplo: «**bind**», «**instanceOf**». Las comillas están disponibles en casi todas las tipografías así que realmente no hay excusa para no utilizarlas, pero quienes tengan problemas tipográficos pueden reemplazarlas por dos corchetes angulares (<< >>) si fuera necesario.

Véase también uso de la tipografía.

compartimento

División gráfica de un símbolo cerrado; por ejemplo un símbolo de clase se divide verticalmente en rectángulos más pequeños. Cada compartimento muestra las propiedades del elemento que representa. Los compartimentos pueden ser de tres tipos: fijos, listas y regiones.

Véase también clase, clasificador.

Notación

Un *compartimento fijo* tiene un formato fijo de partes gráficas y de texto para representar un conjunto fijo de propiedades. El formato depende del tipo de elemento. Un ejemplo sería un compartimento de nombre de clase, que contiene un símbolo de estereotipo y/o nombre, un nombre de clase, y una cadena de propiedades que muestra varias propiedades de la clase. Dependiendo del elemento, puede suprimirse alguna información.

Un *compartimento lista* contiene una lista de cadenas que codifican las partes que constituyen el elemento. Un ejemplo sería una lista de atributos. Los elementos de la lista se pueden mostrar en su orden natural dentro del modelo u ordenados por una o más propiedades (en cuyo caso, el orden natural no sería visible). Por ejemplo, una lista de atributos podría ordenarse primero por visibilidad y en segundo lugar por nombre. Las entradas de la lista pueden mostrarse o suprimirse basándose en las propiedades de los elementos del modelo. Un compartimento de atributos, por ejemplo, podría mostrar únicamente atributos públicos. Se pueden aplicar estereotipos y palabras clave a los constituyentes individuales colgándolos de la correspondiente entrada de la lista. Se pueden aplicar estereotipos y palabras clave a todos los constituyentes que siguen a uno dado situándolos sobre la lista de entradas, de forma que afecten a todas las entradas subsiguientes de la lista hasta el final de la misma o hasta que se encuentre otra declaración similar. La cadena «**constructor**» situada en una línea separada en una lista de operaciones provocaría que se aplicara el estereotipo de constructor a todas las operaciones subsiguientes, pero si apareciera la cadena «**query**» más adelante en la lista, revocaría la primera declaración y la reemplazaría por el estereotipo «**query**».

Una *región* es un área que contiene una subimagen gráfica que muestra una subestructura del elemento, a menudo potencialmente recursivo. Un ejemplo sería una región de estado anidado. La naturaleza de la subimagen es característica de cada elemento del modelo. Se pueden incluir en un mismo símbolo regiones y compartimentos de texto, pero puede resultar

compartimento
de atributos

compartimento
de operaciones

compartimento
definido por el usuario

ReguladorDeComercio
tiempoEspera: Tiempo límite: Real
suspenderComercio(tiempo: Tiempo) reanudarComercio()
señales crackDelMercado(cantidad: Real)

nombre del
compartimento
elemento de la lista

Figura 13.48 Compartimento con nombre en una clase

desordenado. Las regiones se utilizan frecuentemente en elementos recursivos, mientras que el texto se emplea en elementos hoja sin subestructura recursiva.

Una clase tiene tres compartimentos predefinidos: nombre, que es un compartimento fijo; atributos, que es un compartimento lista; y operaciones, que es otro compartimento lista. El que realiza el modelo puede añadir otro rectángulo y situar su nombre en la parte superior del compartimento con un tipo de letra distintivo (por ejemplo en pequeño y en negrita).

La sintaxis gráfica depende del elemento y del tipo de compartimento. La Figura 13.48 muestra un compartimento para señales. La Figura 13.166 muestra un compartimento para responsabilidades.

compendio

Resultado de filtrar, combinar y abstraer las propiedades de un conjunto de elementos dentro de su contenedor para dar una visión más abstracta, de más alto nivel, de un sistema.

Véase paquete.

Semántica

Los contenedores, tales como los paquetes y las clases, pueden tener propiedades y relaciones derivadas que resuman las propiedades y relaciones de su contenido. Esto permite al creador de modelos tener una mejor comprensión del sistema en un nivel más alto y con menos detalle, que resulta más fácil de comprender. Por ejemplo, una dependencia entre dos paquetes indica que existe una dependencia entre al menos una pareja de elementos de los dos paquetes. El compendio tiene menos detalle que la información original. Puede haber una pareja de dependencias individuales, o muchas parejas, que están representadas por dependencias del nivel de paquetes. En todo caso, el creador del modelo sabe que un cambio efectuado en un paquete *puede* afectar al otro. Si se necesitan más detalles, siempre se puede examinar detalladamente el contenido una vez que se ha observado el compendio de alto nivel.

Análogamente, una dependencia de uso entre dos clases suele indicar una dependencia entre sus operaciones, tal como el caso de un método de una clase que llama a una operación (¡No a un método!) de otra clase. Muchas dependencias del nivel de clases se derivan de dependencias entre operaciones o atributos.

En general, las relaciones que se resumen en un contenedor indican la existencia de al menos una aparición de la relación entre contenidos. Normalmente, no indican que todos los elementos contenidos participen en la relación.

componente

Una parte física reemplazable de un sistema que empaqueta su implementación, y es conforme a un conjunto de interfaces a las que proporciona su realización.

Semántica

Un componente representa una pieza física de la implementación de un sistema, incluyendo el código del software (fuente, binario, o ejecutables) o equivalentes, tales como guiones o archivos de comandos. Algunos componentes tienen identidad y pueden poseer entidades físicas, que incluyen objetos en tiempo de ejecución, documentos, bases de datos, etcétera. Los componentes existen en el dominio de la implementación son unidades físicas en los computadores que se pueden conectar con otros componentes, sustituir por componentes equivalentes, trasladar, archivar, etcétera. Los modelos pueden representar dependencias entre componentes, tales como dependencias del compilador y de tiempo de ejecución o dependencias de la información en una organización humana. Una instancia de un componente se puede utilizar para representar las unidades de implementación que existen en tiempo de ejecución, incluyendo su localización en instancias de un nodo.

Los componentes tienen dos características. Empaquetan el código que implementa la funcionalidad de un sistema, y algunas de sus propias instancias de objetos que constituyen el estado del sistema. Los llamamos últimos componentes de la identidad, porque sus instancias poseen identidad y estado.

Código. Un componente contiene el código para las clases de implementación y otros elementos. (La palabra código se interpreta de forma amplia e incluye los guiones, las estructuras de hipertexto, y otras formas de descripciones ejecutables.) Un componente de código fuente es un paquete para el código fuente de las clases de implementación. Algunos lenguajes de programación (tales como C++) distinguen archivos de declaración de los archivos de método, pero todos son componentes. Un componente de código binario es un paquete para el código compilado. Una biblioteca del código binario es un componente.

Un componente ejecutable contiene código ejecutable. Cada tipo de componente contiene el código para las clases de implementación que realizan algunas clases e interfaces lógicas. La relación de realización asocia un componente con las clases y las interfaces lógicas que implementan sus clases de implementación. Las interfaces de un componente describen la funcionalidad que aporta. Cada operación en la interfaz debe hacer referencia eventualmente a un elemento de la implementación disponible en el componente.

La estructura estática, ejecutable de una implementación de un sistema se puede representar como un conjunto interconectado de componentes. Las dependencias entre componentes significan que los elementos de la implementación en un componente requieren los servicios de los elementos de la implementación en otros componentes. Tal uso requiere que los elementos del proveedor sean públicamente visibles en sus componentes. Un componente puede también

tener elementos privados, pero éstos no pueden ser proveedores directos de servicios para otros componentes.

Los componentes pueden ser contenidos por otros componentes. Un componente contenido es sólo otro elemento de la implementación dentro de su contenedor.

Una instancia de componente es una instancia de un componente en una instancia del nodo. Las instancias de componentes de código fuente y de código binario pueden residir en instancias particulares de un nodo, pero las instancias de componentes ejecutables son las más útiles. Si una instancia de componente ejecutable está situada en una instancia de un nodo, los objetos de las clases de implementación presentes en el componente pueden ejecutar operaciones cuando están situados en la instancia del nodo. Si no, un objeto situado en una instancia de un nodo no puede ejecutar operaciones y se debe mover o copiar a otra instancia del nodo para ejecutar dichas operaciones.

Si un componente carece de identidad, todas sus instancias son iguales. No importa cuál de ellas esté ejecutando un objeto. Todos se comportan igual. Como no tienen ninguna identidad, las instancias de componentes no tienen valores o estado por sí mismas.

Identidad. Un componente de identidad tiene identidad y estado. Posee los objetos físicos, que están situados en él (y, por lo tanto, en la instancia del nodo que contiene la instancia del componente). Puede tener atributos, relaciones de composición con los objetos poseídos, y asociaciones a otros componentes. Desde este punto de vista, es una clase. Sin embargo, la totalidad de su estado debe hacer referencia a las instancias que contiene. Eso es lo que le hace un componente, en lugar de una clase ordinaria. Una clase de implementación se utiliza a menudo para representar el componente en su totalidad. Esto se llama clase dominante y se compara a menudo con el componente mismo, aunque no son la misma cosa.

Un objeto que solicita servicios de un componente de identidad debe seleccionar una instancia específica del componente, generalmente teniendo una asociación a uno de los objetos poseídos por el componente. Debido a que cada instancia de componente de identidad tiene estado, las distintas peticiones a instancias pueden producir resultados diferentes.

Ejemplo

Por ejemplo, un corrector ortográfico puede ser un componente. Si tiene un diccionario fijo, puede ser modelado como un componente sin identidad. Todas las instancias de él producen los mismos resultados y no hay memoria de peticiones anteriores. Si el diccionario puede ser actualizado, entonces debe ser modelado como un componente de identidad.

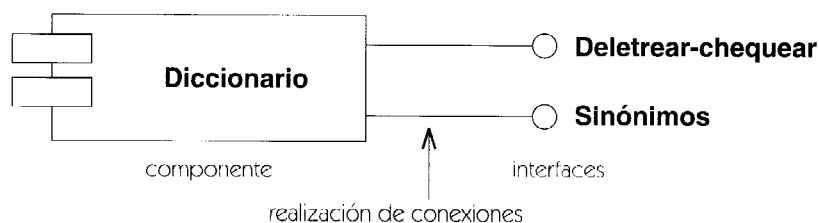


Figura 13.49 Componente

Puede haber diversas versiones del diccionario, que corresponden a diversas instancias del componente del corrector ortográfico. Un solicitante debe dirigir su petición a una instancia específica del componente. El componente destino de la petición es a menudo implícito dentro de un contexto particular, pero esto es una opción que debe ser parte del diseño.

Estructura

Un componente *ofrece* un conjunto de elementos de implementación, tales como clases de implementación. Esto significa que el componente proporciona el código para los elementos. Varios componentes pueden ofrecer un elemento de implementación dado.

Un componente puede tener operaciones e interfaces, que deben ser implementados por sus elementos de implementación.

Un componente de identidad es un *contenedor* físico para las entidades físicas, tales como objetos de tiempo de ejecución y bases de datos. Para proporcionar manejadores para sus elementos contenidos, puede tener los atributos y asociaciones salientes, que deben ser implementados por sus elementos de implementación. Un componente de identidad puede señalar una clase dominante que ofrezca todos sus atributos y operaciones públicas, pero tal clase es uno más de sus elementos de implementación.

Notación

Un componente se representa como un rectángulo con dos rectángulos más pequeños que sobresalen de un lado. El nombre del tipo del componente se pone dentro (Figura 13.49). Una instancia de componente tiene un nombre individual separado de un nombre del tipo de componente por dos puntos. La cadena del nombre se subraya para distinguirla de un tipo de componente (Figura 13.50). Un símbolo de instancia de componente se puede dibujar dentro del símbolo de un nodo para indicar que la instancia del componente está localizada en la instancia de un nodo (Figura 13.51).

Si el componente no tiene identidad, el nombre de la instancia se omite generalmente. Los objetos poseídos por una instancia de componente de identidad se pueden dibujar dentro de él. Un componente que contiene atributos u objetos es automáticamente un componente de identidad.

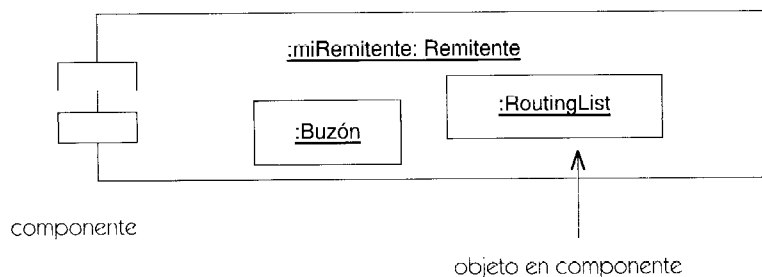


Figura 13.50 Instancias de componentes de identidad con objetos residentes

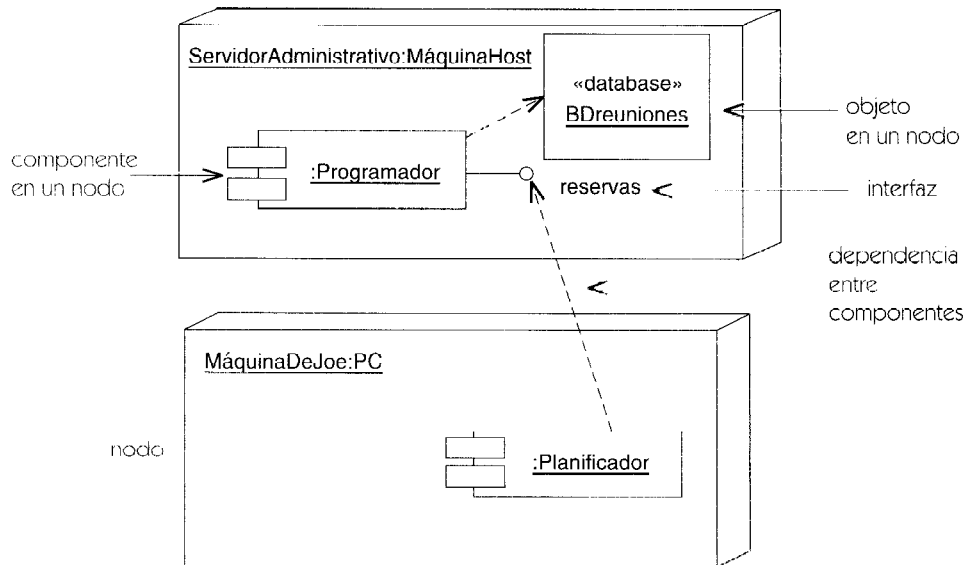


Figura 13.51 Instancias de componentes en nodos

Una clase dominante incluye la interfaz del componente. Puede ser dibujada como clase con un símbolo de componente en la esquina superior derecha como un icono de estereotipo.

En este caso, el componente y su clase dominante comparten la misma interfaz, y cualquier objeto contenido en el componente es accesible desde la clase dominante por los enlaces de composición. La Figura 13.52 presenta un ejemplo.

Las operaciones y las interfaces disponibles para los objetos exteriores se pueden representar directamente en el símbolo de clase. Éstos son su comportamiento como clase. Los contenidos del subsistema se representan en un diagrama separado. Cuando la característica de un subsistema de ser instanciable no necesita ser representada, se puede utilizar la notación ordinaria del paquete.

Las dependencias de un componente con otros componentes o elementos del modelo se representan usando líneas discontinuas con las puntas de flecha en los elementos del proveedor (Figura 13.53). Si un componente es la realización de una interfaz, se puede utilizar la notación taquigráfica de un círculo unido al símbolo del componente por un segmento de línea. Realizar una interfaz implica, por lo menos, que los elementos de la implementación en el componente proporcionen todas las operaciones de la interfaz. Si un componente utiliza una interfaz de otro

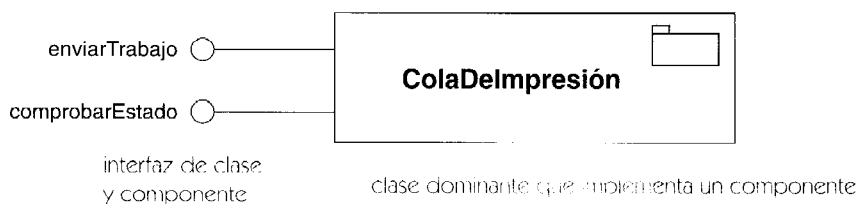


Figura 13.52 Clase dominante para un componente

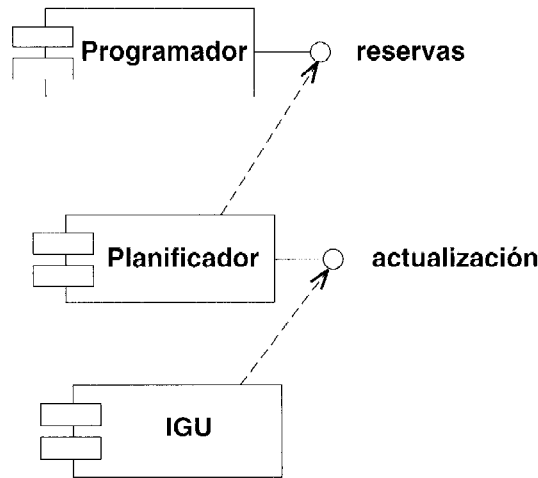


Figura 13.53 Dependencias entre componentes

elemento, la dependencia se puede representar por una línea discontinua con una punta de flecha en el símbolo de la interfaz. Usar una interfaz implica que los elementos de la implementación en el componente no requieren más operaciones de un componente proveedor que las que están enumeradas en el interfaz (pero también el cliente puede depender de otras interfaces).

Discusión

La siguiente definición ampliada explica la intención subyacente en un componente y las consideraciones implicadas en decidir si una parte de un sistema debe considerarse un componente.

- Un componente no es trivial. Es funcional y conceptualmente más grande que una sola clase o una sola línea del código. Típicamente, un componente abarca la estructura y comportamiento de una colaboración de clases.
- Un componente es casi independiente de otros componentes. Aunque raramente se encuentra solo. Más bien, un componente dado colabora con otros componentes y en ese empeño, asume un contexto arquitectónico.
- Un componente es una parte reemplazable de un sistema. Es sustituible, permitiendo sustituirlo por otro que se amolde a las mismas interfaces. El mecanismo de insertar o de sustituir un componente para formar un sistema ejecutable es típicamente transparente al usuario del componente, y es habilitado por los modelos de objetos que requieren pocas o ninguna transformación o por las herramientas que automatizan el mecanismo.
- Un componente satisface una función clara y es lógica y físicamente cohesivo. Denota así un pedazo significativo bien estructuralmente o bien por el comportamiento, para un sistema más grande.
- Un componente existe en el contexto de una arquitectura bien definida. Representa un bloque constructivo fundamental en el que puedan ser diseñados y compuestos los sistemas.

Esta definición es recurrente. Un sistema en un nivel de abstracción puede simplemente ser un componente en un nivel más alto de la abstracción.

- Un componente nunca se encuentra solo. Todo componente presupone una arquitectura y/o el contexto tecnológico en el cual se piensa utilizar.
- Un componente está formado por un conjunto de interfaces. Un componente que se amolda a una interfaz satisface el contrato especificado por la interfaz y se puede sustituir en cualquier contexto en el que se aplique esa interfaz.

Elementos estándar

document, executable, file, library, location, table.

comportamiento

Efecto observable de una operación o evento que incluye sus resultados.

composición

Una forma de asociación de agregación con fuerte sentido de posesión y tiempo de vida coincidente de las partes con el conjunto. Una pieza puede pertenecer a solamente una composición. Las partes con multiplicidad no fija, se pueden crear después del elemento compuesto. Pero una vez creadas, viven y mueren con él (es decir, comparten tiempo de vida). Tales piezas se pueden también quitar explícitamente antes de la muerte del elemento compuesto. La composición puede ser recurrente.

Véase también agregación, asociación, objeto compuesto.

Semántica

Hay una forma de asociación de agregación muy fuerte llamada composición. Una composición es una asociación de agregación con las restricciones adicionales de que un objeto sólo puede ser parte de un compuesto a la vez y que el objeto compuesto es el único responsable de la disponibilidad de todas sus partes. Como consecuencia de la primera restricción, el conjunto de todas las relaciones de composición (incluidas *todas* las asociaciones con la propiedad de la composición) forma un bosque de árboles hechos de objetos y de enlaces de composición. Una parte compuesta no se puede compartir por dos objetos compuestos. Esto concuerda con la intuición normal de la composición física por partes —una no puede ser parte directa de dos objetos (aunque puede indirectamente ser parte de objetos múltiples en diversos niveles de granularidad en el árbol)—.

Por tener la responsabilidad de la disponibilidad de sus partes, entendemos que el elemento compuesto es responsable de su creación y destrucción. En términos de implementación, es responsable de su asignación de memoria. Durante su instanciación, un elemento compuesto debe asegurarse de que hayan sido instanciadas y unidas correctamente a él todas sus partes. Puede crear una pieza por sí mismo o puede asumir la responsabilidad de una pieza existente.

Pero durante la vida del elemento compuesto, ningún otro objeto puede tener esta responsabilidad sobre ellos. Esto significa que el comportamiento para una clase compuesta se puede diseñar con el conocimiento de que ninguna otra clase destruirá o desasignará sus piezas. Un elemento compuesto puede agregar piezas adicionales durante su vida (si lo permite la multiplicidad), siempre que asuma una responsabilidad única sobre ellas. Puede quitar piezas, siempre que la multiplicidad lo permita y la responsabilidad de ellas sea asumida por otro objeto. Si se destruye el elemento compuesto, debe o destruir todas sus piezas o bien dar la responsabilidad de ellas a otros objetos.

Esta definición abarca la mayoría de las intuiciones lógicas e implementaciones comunes de la composición. Por ejemplo, un registro que contiene una lista de valores es una implementación común de un objeto y de sus atributos. Cuando se asigna el registro, la memoria para los atributos se asigna automáticamente también, pero los valores de los atributos pueden necesitar ser inicializados. Mientras existe el registro, ningún atributo se puede eliminar tampoco. Cuando se desasigna el registro, la memoria de los atributos se desasigna también. Ningún otro objeto puede afectar a la asignación de algún atributo dentro del registro. Las características físicas de un registro hacen cumplir los requisitos de un elemento compuesto.

Esta definición de la composición funciona bien con la recolección de basura. Si se destruye el elemento compuesto en sí mismo, el único indicador a la pieza se destruye y la pieza se convierte en inaccesible y está sujeta a la recolección de basura. Sin embargo, la recuperación de piezas inaccesibles es simple incluso con la recolección de basura, lo cual es una razón para distinguir la composición de otra agregación.

Observe que una pieza no necesita ser implementada como una parte física de bloque de memoria con el elemento compuesto. Si la pieza está aparte del elemento compuesto, entonces el compuesto tiene la responsabilidad de asignar y de desasignar la memoria para la pieza, según lo necesita. En C++, por ejemplo, los constructores y destructores facilitan la implementación de elementos compuestos.

Un objeto puede ser parte solamente de un objeto compuesto a la vez. Esto no imposibilita a una clase para ser una parte compuesta de más de una clase en diferentes momentos o en diferentes instancias, pero solamente puede existir un enlace de composición al mismo tiempo para un objeto. Es decir hay una restricción OR entre los posibles compuestos a los que una pieza pudo pertenecer. Un objeto también puede ser parte de diversos objetos compuestos durante su vida, pero solamente uno a la vez.

Estructura

La propiedad de agregación en un extremo de la asociación puede tener los siguientes valores.

ninguno	El clasificador unido no es un agregado o un compuesto.
agregado	El clasificador unido es un agregado. El otro extremo es una pieza.
compuesto	El clasificador unido es un compuesto. El otro extremo es una pieza.

Un extremo de una asociación debe tener por lo menos el valor **ninguno**.

Notación

La composición se representa por un adorno que consiste en un diamante sólido en el extremo de la asociación unida al elemento compuesto (Figura 13.54). La multiplicidad se puede mostrar de manera normal. Debe ser 1 ó 0..1.

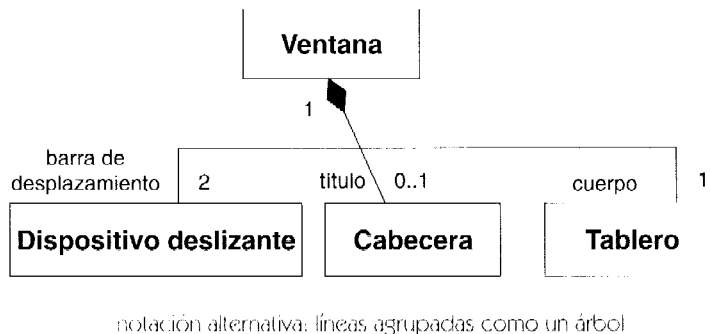
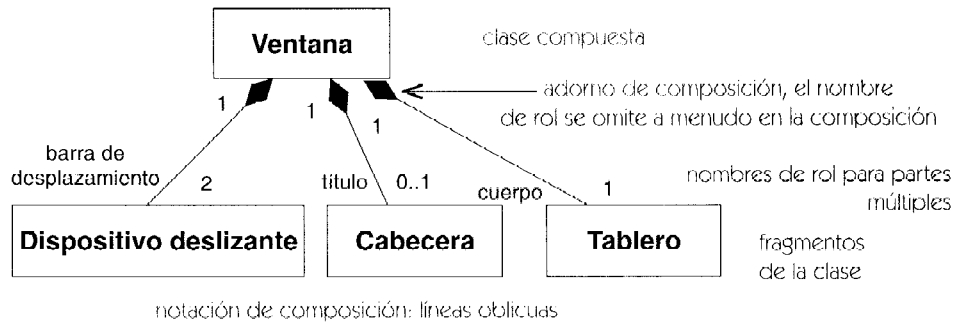


Figura 13.54 Notación de la composición

Alternativamente, la composición puede ser representada gráficamente anidando los símbolos de las partes dentro del símbolo del elemento compuesto (Figura 13.55). Un clasificador anidado puede tener multiplicidad dentro de su elemento compuesto. La multiplicidad se

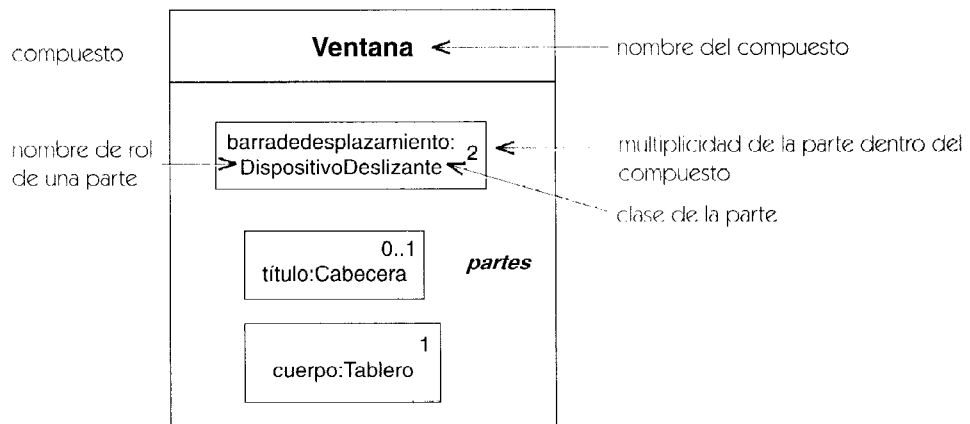


Figura 13.55 Composición como anidamiento gráfico

representa por una cadena de la multiplicidad en la esquina superior derecha del símbolo para la pieza. Si se omite la marca de la multiplicidad, la multiplicidad por defecto son muchas. Un elemento anidado puede tener un nombre de rol dentro de la composición. El nombre se muestra delante de su tipo con la sintaxis

nombre-de-rol : nombre-de-clase

El nombre de rol es el nombre de rol en una asociación implícita de composición del compuesto a la pieza.

Una asociación dibujada enteramente dentro de un borde del compuesto se considera que es parte de la composición. Cualquier objeto conectado por un solo enlace de la asociación debe pertenecer al mismo compuesto. Una asociación dibujada de modo que su línea rompa la frontera del elemento compuesto no se considera parte de la composición. Cualquier objeto en un solo enlace de la asociación puede pertenecer al mismo o a los diversos compuestos (Figura 13.56).

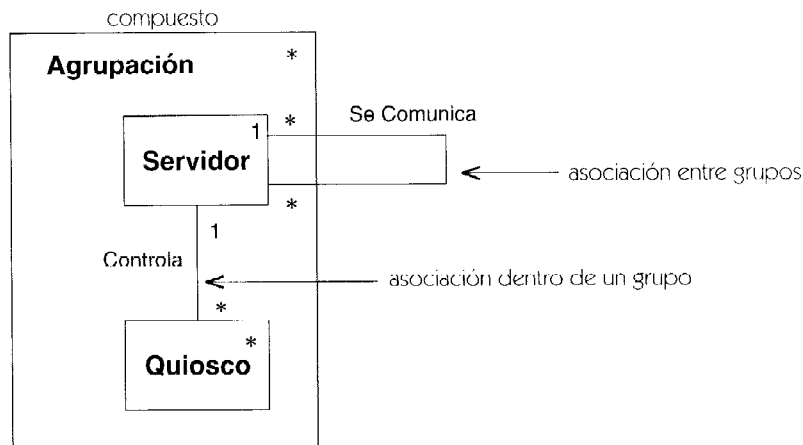
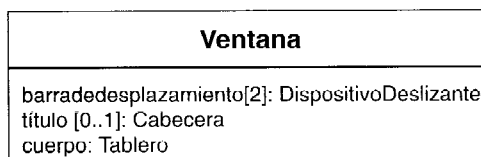


Figura 13.56 Asociación en y entre componentes

Observe que los atributos son, en efecto, relaciones de la composición entre una clase y las clases de sus atributos (Figura 13.57). En general, sin embargo, los atributos deben ser reservados para los valores primitivos de los datos (tales como números, cadenas, y fechas) y no las referencias a las clases, porque ninguna otra relación de las clases que son parte pueden considerarse en la notación de los atributos.

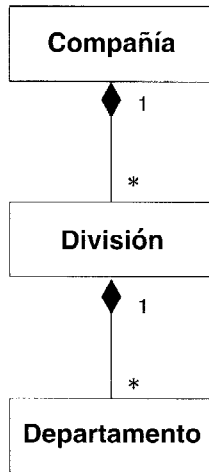


Evitar el uso de atributos para objetos a menos que sean no compartidos y basados en la implementación

Figura 13.57 Los atributos son una forma de composición

Observe que la notación para la composición se asemeja a la notación para la colaboración. Una composición se puede pensar como una colaboración en la cual todos los participantes sean parte de un solo objeto compuesto.

La Figura 13.58 muestra composición a múltiples niveles.



La composición es transitiva:

Departamento es una parte compuesta indirecta de Compañía

Figura 13.58 Composición multinivel

Discusión

(Véase también la discusión en la agregación para las pautas sobre cuándo son apropiadas la agregación, la composición, y la asociación simple.)

La composición y la agregación son metarrelaciones —superan asociaciones individuales para imponer restricciones en el sistema de asociaciones completo—. La composición es significativa a través de relaciones de composición. Un objeto puede tener en la mayoría un enlace de composición (a un elemento compuesto) aunque puede ser que potencialmente venga de más de una asociación de composición. El gráfico completo de la composición y los enlaces y los objetos de la agregación debe ser acíclico, incluso si los enlaces vienen de diversas asociaciones. Observe que estas restricciones se aplican a la instancia del dominio —las asociaciones de agregación forman ciclos a menudo por ellas mismas, y las estructuras recurrentes requieren siempre ciclos de asociaciones.

Considere el modelo en la Figura 13.59. Cada **Autenticación** es una parte compuesta de exactamente una **Transacción**, que puede ser una **Compra** o una **Venta**. Sin embargo no toda **Transacción** necesita tener una **Autenticación**. De este fragmento, tenemos bastante información para concluir que una **Autenticación** no tiene ninguna otra asociación de la composición. Cada objeto de autenticación debe ser parte de un objeto transacción (la multiplicidad es uno); un objeto puede ser parte en de un compuesto como máximo (por la definición); es ya parte de un compuesto (según el dibujo); por lo tanto una **Autenticación** no debería ser parte de ninguna otra asociación de composición. No hay peligro que una **Autenticación** pudiera tener que manejar su propio almacenaje. Una transacción siempre está disponible para tomar esa responsabilidad, aun-

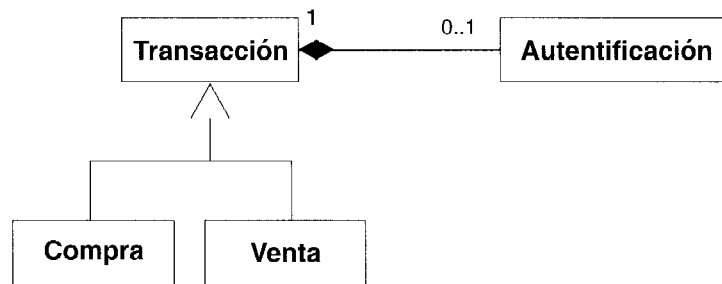


Figura 13.59 Composición a una clase abstracta compuesta

que no todas las **Transacciones** tienen las **Autenticaciones** que necesita manejar. (Por supuesto, la **Autenticación** puede manejarse por sí misma si el diseñador lo desea.)

Ahora considere la Figura 13.60. Un **Autógrafo** puede opcionalmente ser parte de una **Transacción** o de una **Carta**. No puede ser parte de ambos al mismo tiempo (por las reglas de la composición). Este modelo no evita que un **Autógrafo** comience como parte de una carta y se convierta en parte de una **Transacción** (en cuyo caso, debe dejar de ser parte de la **Carta**). De hecho, un **Autógrafo** no necesita ser parte de ninguna cosa. También, de este fragmento modelo, no podemos impedir la posibilidad que el **Autógrafo** es opcionalmente parte de una cierta clase que no se muestra en el diagrama o que se pudo agregar más adelante.

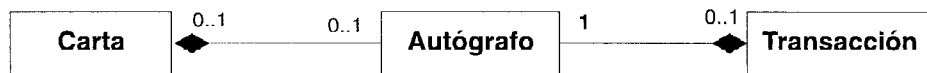


Figura 13.60 Partes compartidas de clases

¿Qué ocurre si es necesario indicar que cada **Autógrafo** debe ser parte de una **Carta** o una **Transacción**? Entonces el modelo se debe replantear, como en la Figura 13.59.

Se puede agregar una nueva superclase abstracta que incluye a **Carta** y a **Transacción** (llamada **Documento**), y la asociación de composición con **Autógrafo** se puede desplazar a ella desde las clases originales. Al mismo tiempo, la multiplicidad del **Autógrafo** al **Documento** se hace uno.

Hay un problema de menor importancia con este acercamiento: La multiplicidad del **documento** al **Autógrafo** se debe hacer opcional, lo que debilita la inclusión obligatoria original del **Autógrafo** dentro de la **Transacción**. La situación se puede modelar usando la generalización de la asociación de composición por sí misma, como en la Figura 13.61. La asociación de composición entre el **Autógrafo** y la **Transacción** se modela como hijo de la asociación de composición entre el **Autógrafo** y el **Documento**. Pero sus multiplicidades se clarifican para el hijo (nótese que siguen siendo coherentes con las heredadas, así que el hijo es sustituible por el padre). Alternativamente, el modelo original puede ser utilizado agregando una restricción entre las dos composiciones, de las cuales siempre debe mantenerse una.

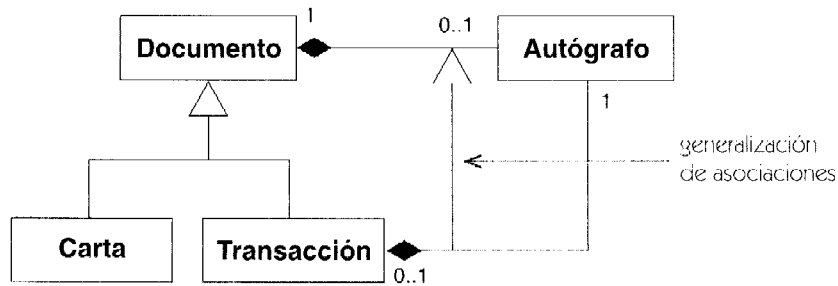


Figura 13.61 Generalización de asociación de composición

concreto

Un elemento generalizable (tal como una clase) que puede ser instanciado directamente. Por lo tanto, su implementación debe ser especificarse completamente. Para una clase, todas sus operaciones deben ser implementadas (por la clase o un antecesor). Antónimo: abstracto.

Véase también clase directa, instanciación.

Semántica

Sólo los clasificadores concretos pueden ser instanciados. Por lo tanto todas las hojas de una jerarquía de generalización deben ser concretas. Es decir, todas las operaciones abstractas y otras propiedades abstractas se deben implementar eventualmente en algún descendiente. (Por supuesto, una clase abstracta puede no tener ningún descendiente concreto, si el programa está incompleto, por ejemplo un marco previsto para la extensión del usuario, pero tal clase no puede utilizarse en una implementación hasta que sus descendientes no sean concretos.)

Notación

El nombre de un elemento concreto aparece en letra normal. El nombre de un elemento abstracto aparece en letra cursiva.

concurrency

El funcionamiento de dos o más actividades durante el mismo intervalo del tiempo. No hay implicación sobre que las actividades estén sincronizadas. En general, funcionan independientemente a excepción de puntos explícitos de la sincronización. La concurrency puede lograrse intercalando o ejecutando simultáneamente dos o más hilos.

Véase transición compleja, estado compuesto, hilo.

concurrency dinámica

Un estado de actividad que representa múltiples ejecuciones concurrentes cuando se activa.

Véase grafo de actividades.

condición de guarda

Es una condición que debe satisfacerse para permitir que se dispare la transición asociada.

Véase también bifurcación, hilo condicional, estado de unión o conjunción, transición.

Semántica

Una condición de guarda es una expresión booleana que forma parte de la especificación de una transición. Cuando se recibe el evento disparador de una transición, ese evento se guarda hasta que la máquina de estados ha finalizado cualquier posible paso que deba ejecutarse hasta finalizar. Entonces se procesa el evento y se evalúa la condición de guarda. Si la condición es verdadera, se permite que se dispare la transición (pero si se activa más de una transición, sólo se dispara una de ellas). La comprobación se produce en el momento en que se procesa el evento disparador. Si la condición de guarda toma el valor falso al ser evaluada cuando se procesa el evento, esa condición no se vuelve a evaluar a no ser que vuelva a producirse el evento disparador, aun cuando la condición pudiera pasar a ser verdadera posteriormente.

Una condición de guarda tiene que ser una consulta, esto es, no puede modificar el valor del sistema o de su estado, ni puede poseer efectos secundarios.

Una condición de guarda puede aparecer en una transición de finalización. En tal caso, seleccionará una de las ramas de la bifurcación.

Notación

La condición de guarda forma parte de la cadena correspondiente a una transición. Adopta el aspecto de una expresión booleana entre corchetes.

[expresión booleana]

Los nombres que se utilicen dentro de la expresión deben estar a disposición de la transición. Serán o bien parámetros del evento disparador o bien atributos del objeto poseedor.

configuración del estado activo

Conjunto de estados activos existente en un momento dado en una máquina de estados. Cuando se dispara una transición se producen cambios en unos cuantos estados del conjunto, pero los demás permanecen inalterados.

Véase activo/a, máquina de estados, transición compleja.

conflicto

Situación en que atributos u operaciones con el mismo nombre se heredan de más de una clase, o cuando el mismo evento permite más de una transición, o cualquier situación similar en la cual las reglas normales produzcan resultados potencialmente contradictorios. Dependiendo de la semántica para cada tipo de elemento del modelo, un conflicto se puede resolver por una regla de resolución del conflicto, puede ser legal producir un resultado no determinista, o se puede indicar que el modelo está mal formado.

Discusión

Es posible evitar conflictos definiéndolos sin tener en cuenta las reglas de la resolución de conflictos, por ejemplo: Si el mismo atributo está definido es más de una superclase, utilice la definición que aparece en superclases anteriores (esto requiere que las superclases estén ordenadas). UML no especifica generalmente las reglas para resolver conflictos, en principio es peligroso contar con tales reglas. Son fáciles de pasar por alto y con frecuencia son el síntoma de problemas más profundos en un modelo.

Es mejor forzar al modelador a ser explícito en lugar de depender de reglas sutiles y posiblemente confusas. En una herramienta o un lenguaje de programación, tales reglas tienen su lugar, únicamente para permitir obtener un significado determinista. Pero sería provechoso que las herramientas proporcionen advertencias cuando se utilicen las reglas de modo que el modelador esté enterado del conflicto.

construcción

Tercera fase de un proceso del desarrollo del software, durante la cual se hace el diseño detallado y el sistema se implementa y se prueban los elementos de software, hardware o circuitería. Durante esta fase, la vista del análisis y la vista del diseño se terminan substancialmente, junto con la mayor parte de la vista de implementación y de parte de la vista del despliegue.

Véase proceso de desarrollo.

constructor

Una operación de ámbito de clase que crea e inicializa una instancia de una clase. Puede ser utilizado como estereotipo de operación.

Véase creación, instanciación.

consulta

Dícese de una operación que proporciona un valor pero no altera el estado del sistema; denota una operación sin efectos secundarios.

contenedor

Un objeto que existe para contener a otros objetos, y que proporciona operaciones para acceder o iterar sobre su contenido, o una clase que describe tales objetos.

Por ejemplo, arrays, listas, y conjuntos.

Véase también agregación, composición.

Discusión

Generalmente es innecesario modelar explícitamente los contenedores. Son la implementación más común para el extremo “muchos” de una asociación. En la mayoría de los modelos, una multiplicidad mayor de uno es suficiente para indicar la semántica correcta. Cuando un modelo de diseño se utiliza para generar código, la clase del contenedor usado para implementar la asociación puede especificarse a un generador de código usando valores etiquetados.

contexto

Una vista de un conjunto de elementos de modelado que están relacionados para un propósito, tal como ejecutar una operación o formar un patrón. Un contexto es un pedazo de un modelo que restringe o proporciona el ambiente para sus elementos. Una colaboración proporciona un contexto para su contenido.

Véase colaboración.

copia

Un tipo de relación de flujo usada en una interacción, en la cual el objeto destino representa una copia del objeto fuente; después de esto ambos son independientes.

Véase también become.

Semántica

La relación de copia es un tipo de relación de flujo que indica la derivación de un objeto desde otro objeto dentro de una interacción. Representa la acción de hacer una copia. Después de que se ejecute un flujo de copia, hay dos objetos independientes cuyos valores pueden evolucionar independientemente.

Una transición de copia dentro de una interacción puede tener un número de secuencia para indicar cuándo ocurre en relación con otras acciones.

Notación

Un flujo de copia se representa por una flecha con línea discontinua desde el objeto original a la nueva copia (en la punta de flecha). La flecha lleva la palabra clave de estereotipo «**copy**» y

puede tener un número de secuencia. Las transiciones de copia pueden aparecer en diagramas de colaboración, de secuencia, y de actividad.

Ejemplo

La Figura 13.62 muestra un archivo que tiene una copia de seguridad en otro nodo. Primero se hace una copia («copy»), después la copia se mueve al nodo secundario («become» con un cambio de localización).

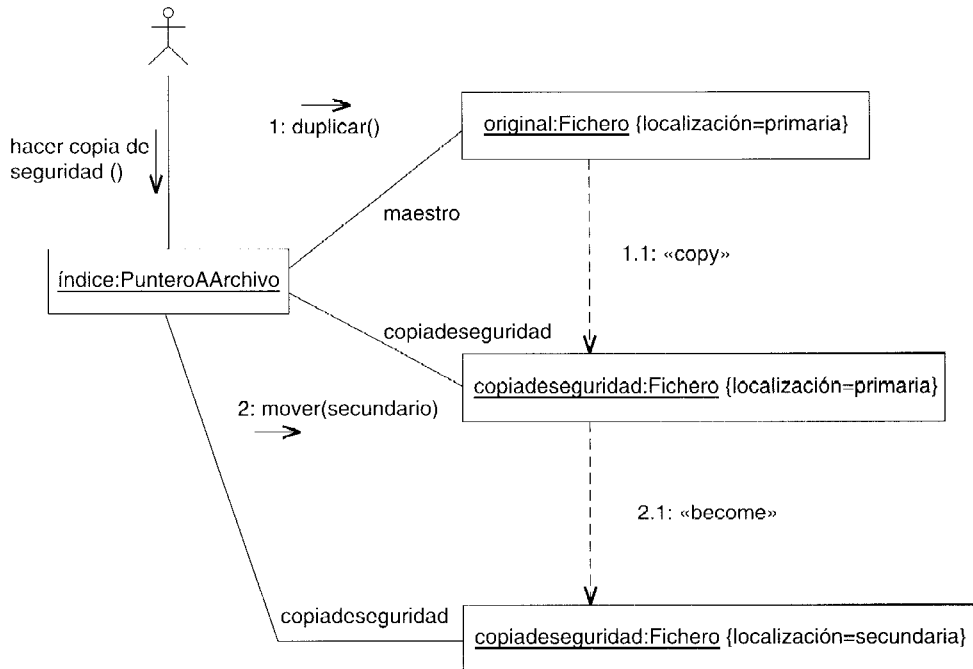


Figura 13.62 Flujo copy y become

creación

La instanciación y la inicialización de un objeto o de otra instancia (como una instancia de un caso de uso). Antónimo: destrucción.

Véase también instanciación.

Semántica

La creación de un objeto es el resultado de un mensaje que instancia el objeto. Una operación de creación puede tener parámetros que se utilicen para la inicialización de la nueva instancia.

Al finalizar la operación de creación, el nuevo objeto obedece a las restricciones de su clase y puede recibir mensajes.

Una operación de creación, o constructor, se puede declarar como operación del ámbito de la clase. El destino de la operación es (al menos conceptual) la propia clase. En un lenguaje de programación como Smalltalk, una clase se implementa como objeto de ejecución real y la creación por lo tanto se realiza como mensaje normal a tal objeto. En un lenguaje como C++, no hay un objeto de ejecución real. La operación se puede pensar como mensaje conceptual que se ha optimizado dejándolo al margen de la ejecución. El enfoque de C++ imposibilita la oportunidad de computar la clase que va a ser instanciada. Si no, cada acercamiento se puede modelar como mensaje enviado a una clase.

Las expresiones del valor inicial para los atributos de una clase son (conceptualmente) evaluados en la creación, y los resultados se utilizan para inicializar los atributos. El código para una operación de creación puede sustituir explícitamente estos valores, así que las expresiones del valor inicial se deben tomar como valores por defecto que pueden ser modificados.

Dentro de una máquina de estados, los parámetros de la operación de construcción que creó el objeto están disponibles como evento actual implícito en la transición que sale del estado inicial del nivel superior.

Notación

En un diagrama de clases, una operación de creación (constructor) se incluye como una de las operaciones en la lista de operación de la clase. Puede tener una lista de parámetros, pero el valor de vuelta es implícitamente una instancia de la clase y puede omitirse. Como operación del ámbito de la clase, su cadena con el nombre debe estar subrayada (Figura 13.63). Puede tener también el estereotipo «**constructor**».

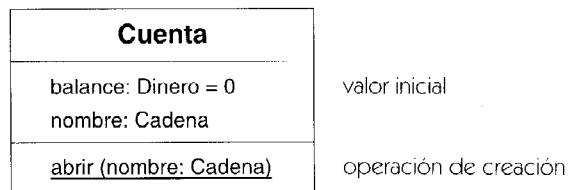


Figura 13.63 Operación de creación

Una ejecución de una operación de creación dentro de un diagrama de secuencia se representa dibujando una flecha de mensaje, con su punta de flecha en el símbolo del objeto (rectángulo con el nombre del objeto subrayado). Debajo del objeto, el símbolo es la cuerda de salvamento para el objeto (línea llena de línea discontinua o doble, dependiendo de si es activa), que continúa hasta la destrucción del objeto o el final del diagrama (Figura 13.64).

La ejecución de una operación de creación, dentro de un diagrama de colaboración, se representa incluyendo un símbolo de objeto con la propiedad {new}. El primer mensaje al objeto es implícitamente el mensaje que crea el objeto. Aunque el mensaje se dirige realmente a la propia clase, este flujo generalmente se omite y se muestra el mensaje inicializando (instanciado, pero sin inicializar) la instancia, tal y como lo muestra la Figura 13.65.

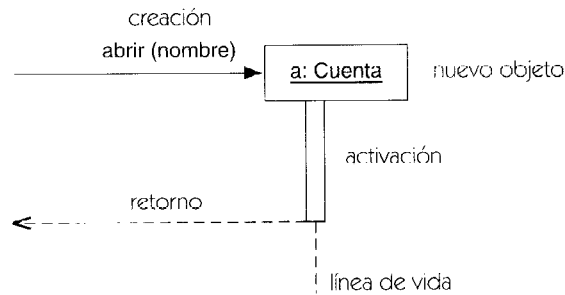


Figura 13.64 Diagramas de secuencia de la creación

Véase también diagrama de colaboración y de secuencia para entender la notación para representar la creación dentro de la implementación de un procedimiento.

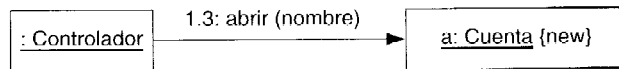


Figura 13.65 La creación en un diagrama de colaboración

delegación

La capacidad de un objeto de emitir un mensaje a otro objeto en respuesta a un mensaje. La delegación puede ser utilizada como alternativa a la herencia. En algunos lenguajes (tales como *Self*), está apoyado por mecanismos de herencia en el propio lenguaje. En la mayoría de los otros lenguajes, tales como C++ y Smalltalk, puede implementarse con una asociación o agregación a otro objeto.

Una operación en el primer objeto invoca una operación en el segundo objeto para lograr realizar su trabajo. Contrasta con: herencia.

Véase también asociación.

dependencia

Una relación entre dos elementos, en los cuales un cambio en un elemento (el proveedor) puede afectar o proveer la información necesaria para el otro elemento (el cliente). Éste es un término de conveniencia que agrupa varios tipos de modelados de relaciones.

Véase relación: Tabla 13-2 para un diagrama completo de las relaciones de UML.

Semántica

La dependencia es una declaración de relación entre dos elementos en un modelo o en modelos diferentes. El término, de forma un poco arbitraria, agrupa varios tipos de relaciones diferentes,

al igual que el termino biológico *invertebrado* agrupa a todos los animales excepto a los *vertebrados*.

En un caso en el cual la relación represente una asimetría del conocimiento, los elementos independientes se llaman proveedores y los elementos dependientes se llaman clientes.

Una dependencia puede tener un nombre para indicar su rol en el modelo. Generalmente, sin embargo, la presencia de la dependencia por sí misma es suficiente para que el significado quede totalmente claro, y el nombre sea redundante. Una dependencia puede tener un estereotipo para establecer la naturaleza precisa de la dependencia, y puede tener un texto descriptivo para explicarlo con más detalle, aunque de manera informal.

Una dependencia entre dos paquetes indica la presencia de, al menos, una dependencia del tipo indicado entre un elemento de cada paquete (excepto para acceso e importación, las cuales están relacionadas con los paquetes directamente). Por ejemplo, la dependencia de uso entre dos clases, se puede representar como una dependencia de uso entre los paquetes que las contienen. Una dependencia entre paquetes no significa que los elementos en el paquete tengan dicha dependencia, de hecho esa situación es poco común.

Véase paquete.

Una dependencia puede contener un conjunto de referencias a dependencias subordinadas. Por ejemplo, una dependencia entre dos paquetes puede hacer referencia a dependencias subyacentes entre clases.

Las dependencias no son necesariamente transitivas.

Observe que la asociación y la generalización encajan dentro de la definición general de dependencia, pero tienen su propia representación y notación en el modelo y no están consideradas generalmente como dependencias. Una relación de realización tiene su propia notación especial, pero se considera una dependencia.

La dependencia tiene diferentes variantes que representen diversos tipos de relaciones: abstracción, ligadura, combinación, permiso, y uso.

Abstracción. Una dependencia de abstracción representa un cambio en el nivel de abstracción de un concepto. Ambos elementos representan el mismo concepto de diversas maneras. Generalmente uno de los elementos es más abstracto, y el otro es más concreto, aunque las situaciones son posibles cuando ambos elementos son representaciones alternativas en el mismo nivel de abstracción. De lo menos específico a las relaciones más específicas, la abstracción incluye los estereotipos *traza*, *refinamiento* (la palabra clave **refine**), *realización* (que tiene su propia notación especial), y *derivación* (la palabra clave **derive**).

Ligadura. Una dependencia de ligadura relaciona un elemento sujeto a una plantilla con la propia plantilla. Los argumentos para los parámetros de la plantilla se añaden a la dependencia de ligadura como una lista.

Permiso. Una dependencia de permiso (representada siempre como un estereotipo específico) relaciona un paquete o una clase con un paquete o una clase a la cual se concede una cierta categoría de permiso para utilizar su contenido. Los estereotipos de la dependencia de permiso son **access**, **friend**, e **import**.

Uso. Una dependencia de uso (palabra clave «use») conecta un elemento del cliente con un elemento del proveedor, cuya modificación puede requerir cambios al elemento cliente. El uso representa a menudo una dependencia de implementación, en la cual un elemento hace uso de los servicios de otro elemento para implementar su comportamiento. Los estereotipos de uso incluyen llamada, instanciación (palabra clave **instantiate**), parámetro, y envío. Esto es una lista abierta. En varios lenguajes de programación pueden existir otros tipos de dependencia de uso.

Notación

Una dependencia se representa mediante una flecha con línea discontinua entre dos elementos del modelo. El elemento de modelo en la cola de la flecha (el cliente) depende del elemento de modelo en la punta de flecha (el proveedor). La flecha se puede etiquetar con una palabra clave opcional, para indicar el tipo de la dependencia, y un nombre opcional (Figura 13.66).

Muchos otros tipos de relaciones usan una flecha de líneas discontinuas con una palabra clave, aunque no entran la definición de dependencia. Éstos incluyen el flujo (conversión y copia), la combinación (extensión e inclusión), y la anexión de una nota o de una restricción al elemento modelo que describe. Si uno de los elementos es una nota o una restricción, la flecha se suprime porque la nota o la restricción es siempre el origen de la flecha.

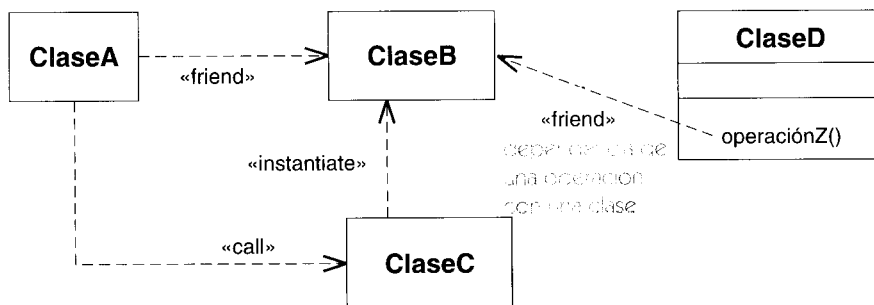


Figura 13.66 Algunas dependencias entre clases

Elementos estándar

become, bind, call, copy, create, derive, extend, friend, import, include, instanceof, instantiate, powertype, send, trace, use.

derivación

Una relación entre un elemento y otro elemento que se puede calcular a partir de aquél. La derivación se modela como un estereotipo de una dependencia de abstracción con la palabra clave **derive**.

Véase elemento derivado.

desarrollo incremental

Denota el desarrollo de un modelo y de otros artefactos de un sistema como una serie de versiones, cada una de las cuales está completa hasta cierto punto en lo tocante a precisión y funcionalidad, pero de tal modo que cada una va añadiendo más detalles a la versión anterior. La ventaja es que cada versión del modelo se puede evaluar y depurar tomando como base los cambios relativamente pequeños efectuados respecto a la versión anterior, lo cual hace más sencillo efectuar correctamente esos cambios. El término está íntimamente relacionado con el concepto de desarrollo iterativo.

Véase proceso de desarrollo.

desarrollo iterativo

Dícese del desarrollo de un sistema mediante un proceso fragmentado en una serie de pasos o iteraciones, cada uno de los cuales proporciona una aproximación mejor que la anterior del sistema deseado. El resultado de cada paso tiene que ser un sistema ejecutable que se pueda ejecutar, comprobar y depurar. El desarrollo iterativo está ligado fuertemente con el concepto de desarrollo incremental. En el desarrollo iterativo e incremental, cada iteración agrega una funcionalidad incremental a la iteración anterior. El orden en que se añade la funcionalidad se selecciona para equilibrar el tamaño de las iteraciones y para atacar desde un principio las fuentes potenciales de riesgo, antes de que se incremente demasiado el coste de resolver los errores.

Véase proceso de desarrollo.

descendiente

Un hijo o un elemento encontrado mediante una cadena de relaciones filiales; el fin transitivo de la relación de especialización. Antónimo: antecesor.

Véase generalización.

descriptor

Un elemento del modelo que describe las propiedades comunes de un conjunto de instancias, incluyendo su estructura, relaciones, comportamiento, restricciones, propósito, etcétera.

Contrastar con: instancia.

Semántica

La palabra *descriptor* caracteriza los elementos del modelo que describen conjuntos de instancias. La mayoría de los elementos en un modelo son descriptores. La palabra incluye todos los elementos del modelo —clases, asociaciones, estados, casos de uso, colaboraciones, etcétera—. A veces, la palabra *tipo* se utiliza con este significado, pero esa palabra se utiliza a menudo en un sentido más restringido para designar cosas como clases. La palabra *descriptor*

pretende incluir cualquier clase de elementos descriptivos. Un descriptor tiene un objetivo y una extensión. La descripción de la estructura y otras reglas generales son el objetivo. Cada descriptor caracteriza un sistema de casos, que son su extensión. No se asume que la extensión sea accesible físicamente en tiempo de ejecución. La dicotomía principal de un modelo es la distinción descriptor-instancia.

Notación

La relación entre un descriptor y sus instancias se refleja usando el mismo símbolo geométrico para ambos, pero subrayando la cadena del nombre de una instancia. Un descriptor tiene un nombre, mientras que una instancia tiene un nombre individual y un nombre de descriptor, separado por dos puntos, y se subraya la cadena del nombre.

descriptor completo

La descripción implícita completa de una instancia directa. El descriptor completo se ensambla implícitamente mediante herencia a partir de todos los antecesores.

Véase también clase directa, herencia, clasificación múltiple.

Semántica

Una declaración de una clase o de otro elemento modelo es, de hecho, solamente una descripción parcial de sus instancias; llámese el *segmento de la clase*. En general, un objeto contiene más estructura que la descrita por el segmento de la clase de su clase directa. El resto de la estructura es obtenido por herencia de las clases de los antecesores. La descripción completa de todos sus atributos, operaciones, y asociaciones se llama descriptor completo. El descriptor completo no es generalmente manifiesto en un modelo o un programa. El propósito de las reglas de la herencia es proporcionar una manera de construir automáticamente el descriptor completo de los segmentos. En principio, hay varias maneras de hacer esto, a menudo llamado *protocolo de metaobjetos*. UML define un conjunto de reglas para la herencia que cubren la mayoría de los lenguajes de programación más populares y son también útiles para modelado conceptual. Sepa, sin embargo, que existen otras posibilidades —por ejemplo el lenguaje CLOS.

despliegue²

Etapas del desarrollo que describe la configuración del sistema para su ejecución en un ambiente del mundo real. Para el despliegue, se deben tomar decisiones sobre los parámetros de la configuración, funcionamiento, asignación de recursos, distribución, y concurrencia. Los resultados de

² El término inglés "deployment" ha recibido diferentes traducciones dependiendo del contexto en que aparece en el conjunto de libros sobre UML. Se traduce por despliegue cuando se trata de diagramas de despliegue aunque a veces se ha utilizado el término "implantación" cuando se trata de la etapa final del desarrollo.

esta fase se capturan en archivos de configuración y también en la vista del despliegue. Ponga en contraste el análisis, el diseño, la implementación, y el despliegue (implantación).

Véase proceso de desarrollo, etapas de modelado.

destrucción

La eliminación de un objeto y la liberación de sus recursos. La destrucción de un objeto compuesto conduce a la destrucción de sus partes compuestas. La destrucción de un objeto no destruye automáticamente los objetos relacionados por la asociación ordinaria o incluso por la agregación, pero cualquier enlace que involucre al objeto se destruye con él.

Véase también composición, estado final, instanciación.

Notación

Véase diagrama de colaboración y de secuencia (Figura 13.70) donde se muestra la notación de destrucción dentro de la implementación de un procedimiento. En un diagrama de secuencia, la destrucción de un objeto es representada por una X grande en la línea de vida del objeto (Figura 13.67). Se coloca en el mensaje que hace que el objeto sea destruido o en el punto donde el objeto finaliza por sí mismo. Un mensaje que destruye un objeto se puede representar con el estereotipo «**destroy**». En un diagrama de colaboración, la destrucción de un objeto durante una interacción se representa por la restricción {**destroyed**} en el objeto. Si el objeto se crea y se destruye en la interacción, se debe usar en su lugar la restricción {**transient**}.

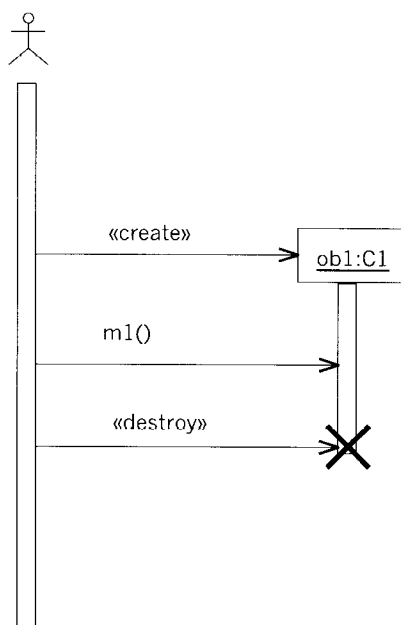


Figura 13.67 Creación y destrucción

destruir

Para eliminar un objeto y liberar sus recursos. Esto es generalmente una acción explícita, aunque puede ser la consecuencia de otra acción o restricción o de la recolección de basura.

Véase destrucción.

diagrama

Una representación gráfica de una colección de elementos del modelo, construida a menudo como un gráfico conexo de arcos (relaciones) y de vértices (otros elementos modelo). UML ofrece diagramas de clases, diagramas de objetos, diagramas de casos de uso, diagramas de secuencia, diagramas de colaboración, diagramas de estados, diagramas de actividad, diagramas de componentes, y diagramas de despliegue.

Véase también información de fondo, uso de la tipografía, hiperenlace, palabra clave, etiqueta, paquete, ruta, elemento de representación, lista de propiedades.

Semántica

Un diagrama no es un elemento semántico. Un diagrama muestra representaciones de elementos semánticos de modelo, pero su significado no se ve afectado por la forma en que son representados.

Un diagrama está contenido dentro de un paquete.

Notación

La mayoría de los diagramas de UML y algunos símbolos complejos son grafos que contienen formas conectadas por rutas. La información está sobre todo en la topología, no en el tamaño o la colocación de los símbolos (hay algunas excepciones, tales como un diagrama de secuencia con un eje métrico de tiempo). Hay tres clases importantes de relaciones visuales: conexión (generalmente de líneas a formas de 2 dimensiones), contención (de símbolos por formas cerradas de 2 dimensiones), y adhesión visual (un símbolo que está “cerca” de otro en un diagrama). Esas relaciones geométricas se reasignan a conexiones entre nodos en un gráfico en la forma analizada de la notación.

La notación de UML está pensada para ser dibujada en superficies bidimensionales. Algunas formas bidimensionales son proyecciones de formas tridimensionales, tales como cubos, pero todavía se representan como iconos en una superficie bidimensional. En un futuro cercano, la disposición y la navegación tridimensional real será posible en máquinas de sobremesa pero no es común en la actualidad.

Hay cuatro clases de construcciones gráficas que se usan en la notación de UML: iconos, símbolos bidimensionales, rutas, y cadenas.

Un icono es una figura gráfica con un tamaño y forma fijos. No se amplía para contener a su contenido. Los iconos pueden aparecer dentro de símbolos de área, como terminadores en las rutas, o como símbolos independientes que puedan o no conectar con las rutas. Por ejemplo, los símbolos para la agregación (un diamante), la dirección de navegación (una punta de flecha), el estado final (un ojo de buey), y la destrucción del objeto (una X grande) son iconos.

Los símbolos de dos dimensiones tienen altura y anchura variables, y pueden ampliarse para permitir otras cosas, tales como listas de cadenas o de otros símbolos. Muchos de ellos están divididos en compartimentos similares o de tipos diferentes. Las rutas se conectan con los símbolos bidimensionales terminando en el límite del símbolo. El arrastrar o suprimir un símbolo bidimensional afecta a su contenido y a cualquier ruta conectada con él. Por ejemplo, los símbolos para la clase (un rectángulo), el estado (un rectángulo redondeado), y la nota (un rectángulo espigado) son símbolos bidimensionales.

Una ruta es una secuencia de segmentos de recta o de curva que se unen en sus puntos finales. Conceptualmente, una ruta es una sola entidad topológica, aunque sus segmentos se pueden manipular gráficamente. Un segmento no debería existir separado de su ruta. Las rutas se unen siempre a otros símbolos gráficos en ambos extremos (no hay rutas colgantes). Las rutas pueden tener terminadores —es decir, iconos que aparecen en una secuencia en el extremo de la ruta y que califican el significado de la ruta—. Por ejemplo, los símbolos para la asociación (línea continua), la generalización (línea continua con un icono de triángulo), y la dependencia (línea discontinua) son rutas.

Las cadenas presentan varias clases de información en una forma “no analizada”. UML asume que cada uso de una cadena en la notación tiene una sintaxis por la cual pueda ser analizada la información del modelo subyacente. Por ejemplo, hay sintaxis para los atributos, las operaciones, y las transiciones. Esta sintaxis es conforme con la extensión mediante herramientas como una opción de representación. Las cadenas pueden existir como el contenido de un compartimento, como elementos en las listas (en cuyo caso la posición en la lista transporta información), como etiquetas unidas a los símbolos o a las rutas, o como elementos independientes en un diagrama. Por ejemplo, los nombres de clases, las etiquetas de transición, las indicaciones de multiplicidad y las restricciones, son cadenas.

diagrama de actividades

Diagrama que muestra un grafo de actividades.

Véase grafo de actividades.

diagrama de caso de uso

Diagrama que muestra las relaciones existentes entre actores y casos de uso dentro de un sistema.

Véase actor, caso de uso.

Notación

Un diagrama de caso de uso es un grafo de actores, un conjunto de casos de uso encerrados por los límites de un sistema (un rectángulo), asociaciones entre los actores y los casos de uso y generalización entre los actores. Los diagramas de casos de uso muestran elementos procedentes del modelo de casos de uso (casos de uso, actores).

diagrama de clases

Un diagrama de clases es una presentación gráfica de la vista estática, que muestra una colección de elementos declarativos (estáticos) del modelo, como clases, tipos, y sus contenidos y relaciones. Un diagrama de clases puede mostrar una vista de un paquete y contener símbolos de paquetes anidados. Un diagrama de clases contiene ciertos elementos materializados de comportamiento, como operaciones, pero cuya dinámica está representada en otros diagramas, como diagramas de estados o diagramas de colaboración.

Véase también clasificador, diagrama de objetos.

Notación

Un diagrama de clases muestra una presentación gráfica de la vista estática. Normalmente son necesarios varios diagramas de clases para mostrar una vista estática completa. Los diagramas de clases individuales no indican necesariamente divisiones del modelo subyacente, aunque las divisiones lógicas, como los paquetes, son fronteras naturales a la hora de formar diagramas.

diagrama de colaboración

Diagrama que muestra interacciones organizadas alrededor de los roles —esto es, ranuras para instancias y sus enlaces dentro de una colaboración—. A diferencia de los diagramas de secuencia, los diagramas de colaboración muestran explícitamente las relaciones entre roles. Por otra parte, un diagrama de colaboración no muestra el tiempo como una dimensión aparte, por lo que resulta necesario etiquetar con números de secuencia tanto la secuencia de mensajes como los hilos concurrentes. Los diagramas de secuencia y los diagramas de colaboración expresan información similar pero de forma diferente.

Véase colaboración, diagrama de secuencia, patrón.

diagrama de componentes

Es un diagrama que muestra las organizaciones y las dependencias entre tipos de componentes.

Semántica

Un diagrama de componentes representa las dependencias entre componentes software, incluyendo componentes de código fuente, componentes del código binario, y componentes ejecutables (Figura 13.53). Un módulo software se puede representar como componente. Algunos componentes existen en tiempo de compilación, algunos existen en tiempo de enlace, y algunos existen en tiempo de ejecución; otros componentes existen en varias de estas ocasiones. Un componente de sólo compilación es aquel que es significativo únicamente en

tiempo de compilación. Un componente de ejecución, en este caso, sería un programa ejecutable.

Un diagrama de componentes tiene solamente una versión con descriptores, no tiene versión con instancias. Para mostrar las instancias de los componentes, se debe usar un diagrama de despliegue.

Notación

El diagrama de componentes muestra clasificadores de componentes, las clases definidas en ellos, y las relaciones entre ellas. Los clasificadores de componentes también se pueden anidar dentro de otros clasificadores de componentes para mostrar relaciones de definición.

Una clase definida dentro de un componente se puede representar dentro de él, aunque para los sistemas de cualquier tamaño, es posible que resulte más conveniente proporcionar una lista de las clases definidas dentro de un componente, en vez de mostrar sus símbolos.

Un diagrama que contiene clasificadores de componentes y clasificadores de nodo se puede utilizar para mostrar las dependencias del compilador, que se representan como flechas con líneas discontinuas (dependencias) de un componente cliente a un componente proveedor del que depende de cierta manera. Los tipos de dependencias son específicos del lenguaje y se pueden representar como estereotipos de las dependencias.

El diagrama también se puede utilizar para mostrar interfaces y las dependencias de llamada entre componentes, usando flechas con líneas discontinuas desde los componentes a las interfaces de otros componentes.

Véase componente para los ejemplos de diagramas de componentes.

diagrama de despliegue

Un diagrama que muestra la configuración de los nodos de proceso y las instancias de componentes y objetos que residen en ellos. Los componentes representan unidades de código en ejecución. Los componentes que no existen como entidades de ejecución (porque se han compilado aparte) no aparecen en estos diagramas; deben aparecer en diagramas de componentes. Un diagrama de despliegue muestra instancias mientras que un diagrama de componentes muestra la definición de los tipos de los componentes por sí mismos.

Véase también componente, interfaz, nodo.

Semántica

La vista de despliegue contiene las instancias de los nodos conectados por los enlaces de comunicaciones. Las instancias de los nodos pueden contener instancias de ejecución, como instancias de componentes y objetos. Las instancias de componentes y objetos también pueden contener otros objetos. El modelo puede mostrar dependencias entre las instancias y sus interfaces, y también puede modelar la migración de entidades entre nodos u otros contenedores.

La vista de despliegue tiene una forma de descriptor y otra de instancia. La forma de instancia (descrita arriba) muestra la localización de las instancias de los componentes específicos en instancias específicas del nodo como parte de una configuración del sistema. Éste es el tipo más común de vista del despliegue. La forma de descriptor muestra qué tipo de componentes pueden subsistir en qué tipo de nodos y qué tipo de nodos se pueden conectar, de forma similar a un diagrama de clases.

Notación

El diagrama del despliegue es una red de símbolos de nodo conectados por líneas que muestran las asociaciones de comunicación (Figura 13.68). Los símbolos de nodo pueden contener instancias de componentes, indicando que el componente vive o se ejecuta en el nodo. Los símbolos de componente pueden contener objetos, indicando que el objeto es parte del componente. Los componentes se conectan con otros componentes con flechas discontinuas de dependencia (posiblemente a través de interfaces). Esto indica que un componente utiliza los servicios de otro componente. Se puede utilizar un estereotipo para indicar la dependencia exacta, si es necesario.

Los diagramas de despliegue son, en gran medida, semejantes a los diagramas de objetos. Generalmente, muestran las instancias individuales del nodo implicadas en un sistema. Es menos común mostrar un diagrama de despliegue que defina los tipos de nodos que existen y sus posibles relaciones con otros tipos de nodos, como en un diagrama de clases.

La migración de componentes de un nodo a otro nodo u objetos de un componente a otro componente se puede representar usando la palabra clave «**become**» en una flecha de línea discontinua. En este caso, el componente o el objeto reside en su nodo o componente sólo parte del tiempo. La Figura 13.146 muestra un diagrama de despliegue en el que un objeto se mueve entre los nodos.

Véase convertirse.

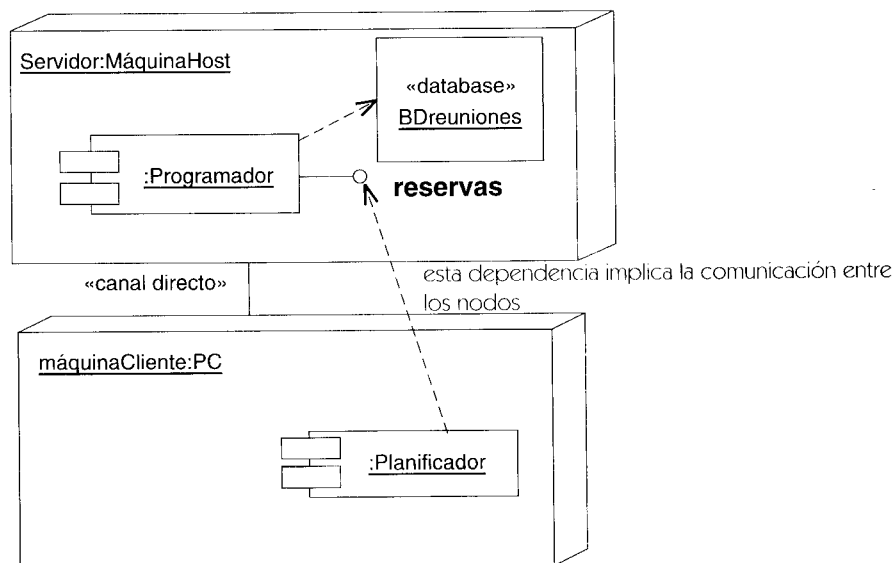


Figura 13.68 Diagrama de despliegue de un sistema cliente-servidor

diagrama de estados

Diagrama que muestra una máquina de estados, incluyendo estados simples, transiciones y estados compuestos anidados. El concepto original fue inventado por David Harel.

Véase máquina de estados.

diagrama de interacción

Se trata de un término genérico que se aplica a varios tipos de diagramas que hacen hincapié en las interacciones entre objetos. Los diagramas de actividades están íntimamente relacionados.

Véase también colaboración, interacción.

Notación

El patrón de interacción entre objetos se muestra en un diagrama de interacción. Los diagramas de interacción tienen diferentes formas, basadas todas ellas en una misma información subyacente pero resaltando cada una un punto de vista de la misma: diagramas de secuencia, diagramas de colaboración y diagramas de actividades.

Un diagrama de secuencia muestra una interacción que está organizada como una secuencia temporal. En particular, muestra los objetos que participan en la interacción mediante sus líneas de vida y mediante los mensajes que intercambian, organizados en forma de una secuencia temporal. Un diagrama de secuencia no muestra los enlaces existentes entre objetos. Los diagramas de secuencia tienen distintos formatos, adecuados para propósitos diferentes.

Un diagrama de secuencia puede existir en forma genérica (describiendo todas las posibles secuencias) y en forma de instancia (describiendo una secuencia de ejecución consistente con la forma genérica). En los casos que carecen de bucles o bifurcaciones, las dos formas son isomorfas: el descriptor es un prototipo de sus instancias.

Un diagrama de colaboración muestra una interacción organizada en torno a los objetos que efectúan operaciones. Es parecido a un diagrama de objetos que muestra los objetos y los enlaces existentes entre ellos que se necesitan para implementar una operación de nivel más elevado.

La secuencia temporal de mensajes se indica mediante los números de secuencia que aparecen en las flechas de flujo de mensajes. Se pueden mostrar secuencias tanto secuenciales como concurrentes, empleando la sintaxis adecuada. Los diagramas de secuencia muestran secuencias temporales empleando el orden geométrico de las flechas que constan en el diagrama. Por tanto, no se necesitan números de secuencia aunque se pueden incluir estos números por comodidad o para permitir el paso a un diagrama de colaboración.

Los diagramas de secuencia y de colaboración expresan una información similar, pero la muestran de maneras diferentes. Los diagramas de secuencia muestran la secuencia explícita de mensajes y son mejores para especificaciones de tiempo real y para escenarios complejos. Los

diagramas de colaboración muestran las relaciones entre objetos y son mejores para comprender todos los efectos que tiene un objeto y para el diseño de procedimientos.

Discusión

Un diagrama de actividades muestra los pasos procedimentales implicados en la realización de una operación de alto nivel. No es un diagrama de interacción, porque muestra el flujo de objeto entre pasos procedimentales y no el flujo de objeto entre objetos. Un diagrama de actividades se centra sobre todo en los pasos del procedimiento. No muestra la asignación de operaciones a clases destino. Un grafo de actividades es una forma de máquina de estados; modela el estado de ejecución de un procedimiento. Existe un cierto número de iconos especiales que se usan en los diagramas de actividades y son equivalentes a las estructuras básicas de UML con ciertas restricciones adicionales, pero se ofrecen por comodidad.

diagrama de objetos

Término que denota los diagramas que muestran los objetos y sus relaciones en un determinado instante de tiempo. Un diagrama de objetos se puede considerar como un caso especial de diagrama de clases en el que se pueden mostrar tanto las clases como las instancias. También están relacionados los diagramas de colaboración, que muestran objetos prototípicos (roles de clasificador) dentro de un contexto.

Véase también diagrama.

Notación

No es necesario que las herramientas admitan un formato separado para los diagramas de objetos. Los diagramas de clases pueden contener objetos, así que un diagrama de clases con objetos pero sin clases es un “diagrama de objetos”. La frase resulta útil, sin embargo, para caracterizar una utilización particular que se puede realizar de distintas formas.

Discusión

Los diagramas de objetos muestran un conjunto de objetos y enlaces que representan el estado de un sistema en un determinado instante de tiempo. Contienen objetos con valores, no descriptores, aunque por supuesto pueden ser prototípicos en muchos casos. Para mostrar un patrón general de objeto y relaciones que se puedan instanciar muchas veces, utilice un diagrama de colaboración, que contiene descriptores (roles de clasificador y roles de asociación) de objetos y enlaces. Si se instancia un diagrama de colaboración, producirá un diagrama de objetos.

El diagrama de objetos no muestra la evolución del sistema con el tiempo. Para este propósito, haga uso de un diagrama de colaboración con mensajes, o bien emplee un diagrama de secuencia para representar una interacción.

diagrama de secuencia

Diagrama que muestra las interacciones entre objetos organizadas en una secuencia temporal. En particular, muestra los objetos participantes en la interacción y la secuencia de mensajes intercambiados.

Véase también activación, colaboración, línea de vida, mensaje.

Semántica

Un diagrama de secuencia representa una interacción, un conjunto de comunicaciones entre objetos organizadas visualmente por orden temporal. A diferencia de los diagramas de colaboración, los diagramas de secuencia incluyen secuencias temporales pero no incluyen las relaciones entre objetos. Pueden existir en forma de descriptor (describiendo todos los posibles escenarios) y en forma de instancia (describiendo un escenario real). Los diagramas de secuencia y los diagramas de colaboración expresan información similar, pero la muestran de maneras distintas.

Notación

Un diálogo de secuencia posee dos dimensiones: la dimensión vertical representa el tiempo; la dimensión horizontal representa los objetos que participan en la interacción (Figura 13.69 y Figura 13.70). En general, el tiempo avanza hacia abajo dentro de la página (se pueden

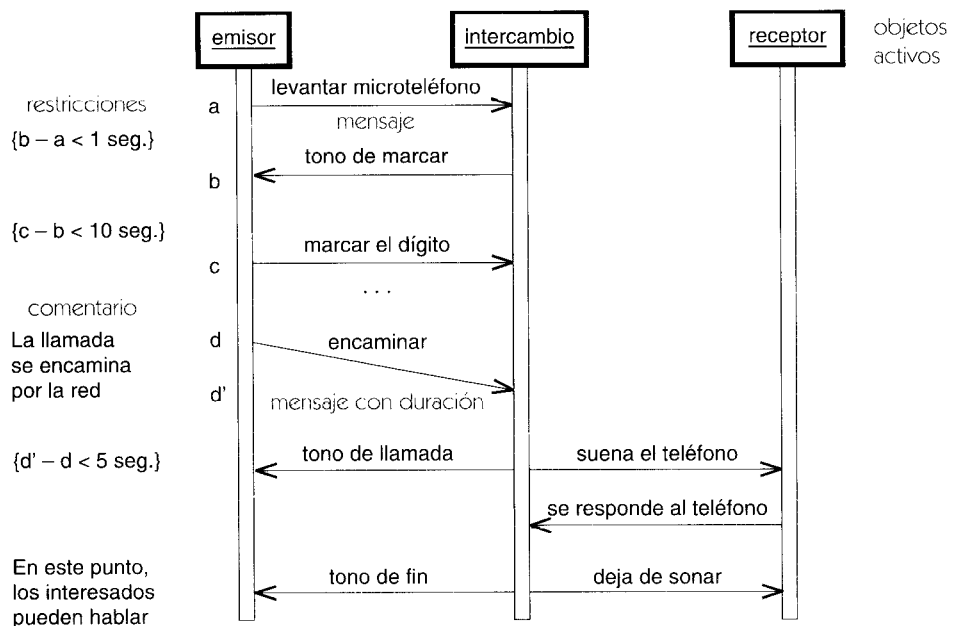


Figura 13.69 Diagrama de secuencia con control asíncrono

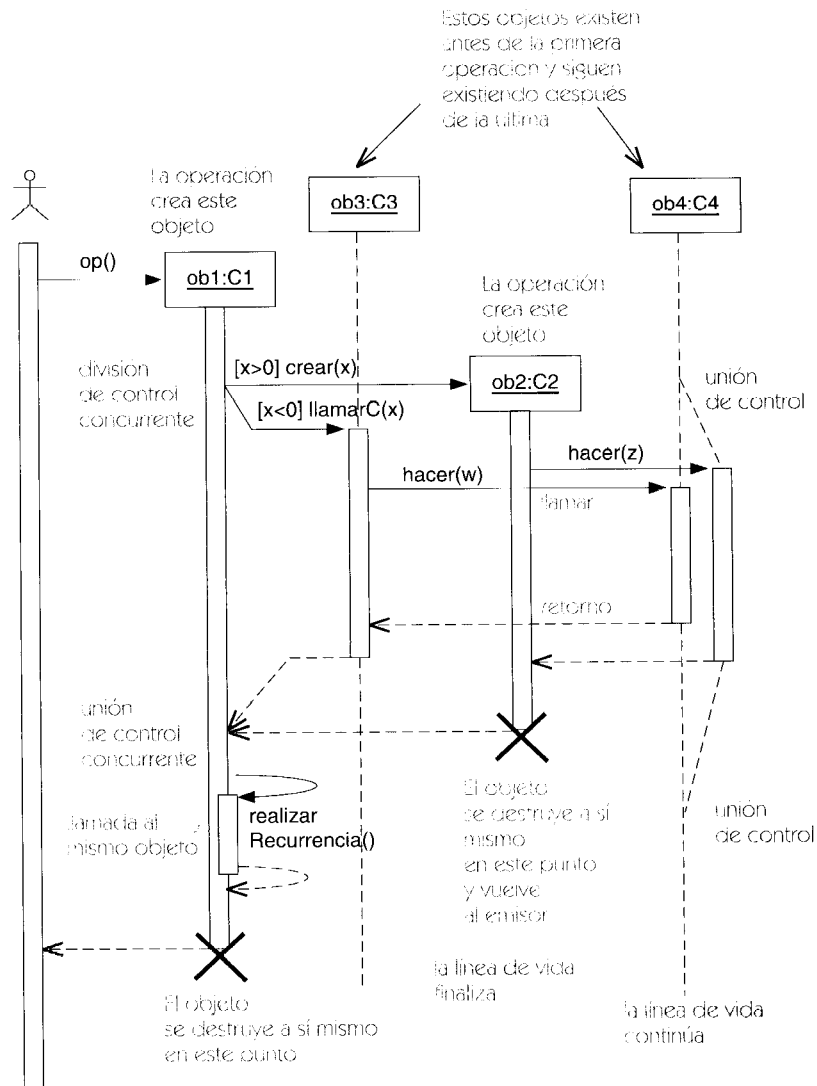


Figura 13.70 Un diagrama de secuencia con flujo de control procedural

invertir los ejes si se desea). Con frecuencia sólo son importantes las secuencias de mensajes pero en aplicaciones de tiempo real el eje temporal puede ser una métrica. La ordenación horizontal de los objetos no tiene ningún significado.

Cada objeto se representa en una columna distinta. Se pone un símbolo de objeto (un rectángulo con el nombre del objeto subrayado) al final de una flecha que representa el mensaje que ha creado el objeto; está situada en el punto vertical que denota el instante en que se crea el objeto. Si un objeto existe desde antes de la primera operación del diagrama, se dibuja el símbolo del objeto en la parte superior del diagrama, antes de todo mensaje. Se dibuja una línea discontinua desde el símbolo de objeto hasta el punto en que se destruye el objeto (si es que esto sucede durante el período de tiempo que muestra el diagrama). Esto se llama línea de vida del objeto. Se pone una X grande en el punto en que deja de existir el objeto o en el punto en que el objeto se destruye a sí mismo. Para cualquier período durante el cual esté activo el

objeto, la línea de vida se amplía para ser una doble línea continua. Esto incluye toda la vida de un objeto activo o las activaciones de un objeto pasivo, un período durante el cual se está ejecutando una operación del objeto, incluyendo el tiempo durante el cual la operación espera al retorno de alguna operación que ella haya invocado. Si el objeto se llama a sí mismo recursivamente, bien sea de manera directa o indirecta, entonces se superpone otra copia de la doble línea continua para mostrar la doble activación (potencialmente, podrían ser más de dos copias). El orden relativo de los objetos no tiene significado, aun cuando resulta útil organizarlos de tal modo que se minimice la distancia que tienen que recorrer las flechas de mensajes. Se puede poner cerca de ella un comentario relativo a la activación.

Cada mensaje se representa mediante una flecha horizontal que va desde la línea de vida del objeto que envió el mensaje hasta la línea de vida del objeto que ha recibido el mensaje. Se puede poner una etiqueta en el margen del lado opuesto a la flecha para denotar el momento en que se envía el mensaje. En muchos modelos se supone que el mensaje es instantáneo, o al menos atómico. Si un mensaje requiere un cierto tiempo para llegar a su destino, entonces la flecha del mensaje se dibuja diagonalmente hacia abajo, de tal modo que el instante de recepción sea posterior al instante de emisión. Los dos extremos pueden tener etiquetas para indicar el momento en que se ha enviado o recibido el mensaje.

Para un flujo de objeto asíncrono entre objetos activos, los objetos se representan mediante líneas dobles continuas y los mensajes se representan como flechas. Se pueden enviar simultáneamente dos mensajes pero no se pueden recibir simultáneamente porque no se puede garantizar una recepción simultánea. La Figura 13.69 muestra un diagrama de secuencia asíncrono.

La Figura 13.70 muestra un flujo de objeto procedimental en un diagrama de secuencia. Cuando se está modelando un flujo de objeto procedimental, un objeto cede el control al producirse una llamada, hasta el subsiguiente retorno. Las llamadas se muestran mediante una punta de flecha continua y rellena. La punta de una flecha de llamada puede hacer que comience una activación o bien un nuevo objeto. Los retornos se muestran con una línea discontinua. El origen de una flecha de retorno puede hacer que finalice una activación o un objeto.

Las bifurcaciones se muestran partiendo la línea de vida del objeto. Cada bifurcación puede enviar y recibir mensajes. Normalmente, cada rama envía mensajes diferentes. Eventualmente, las líneas de vida del objeto tienen que fusionarse de nuevo.

La Figura 13.71 muestra los estados que aparecen durante la vida de una entrada para algún espectáculo. Las líneas de vida pueden verse interrumpidas por un símbolo de estado para mostrar un cambio de estado. Esto corresponde a una transición de conversión en un diagrama de colaboración. Se puede dibujar una flecha hasta el símbolo de estado para indicar el mensaje que ha dado lugar al cambio de estado.

Gran parte de esta notación se ha extraído directamente de la notación Object Message Sequence Chart (Diagrama de Secuencias de Mensajes de Objetos) propuesta por Buschmann, Meunier, Rohnert, Sommerlad y Stal [Buschmann-96], que a su vez se deriva de la notación Message Sequence Chart (diagrama de secuencias de mensajes).

Un diagrama de secuencia también se puede mostrar en forma de descriptor, en el cual los constituyentes son roles en lugar de objetos. Este diagrama muestra el caso general, no una sola ejecución del mismo. Los diagramas del nivel de descriptores se dibujan sin subrayados, porque los símbolos denotan roles y no objetos individuales.

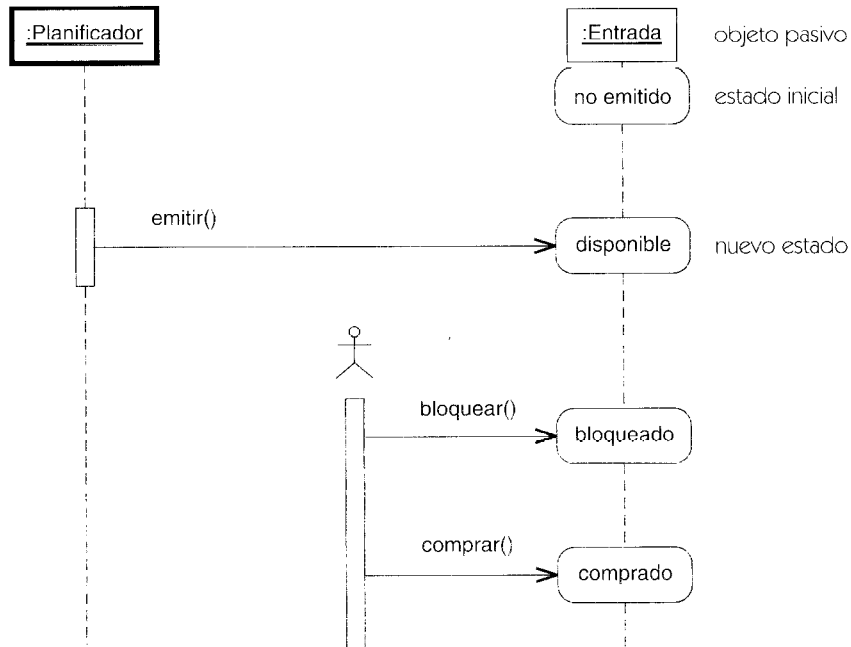


Figura 13.71 Estados de objetos en un diagrama de secuencia

discriminador

Un pseudoatributo que selecciona un elemento hijo de entre un conjunto de hijos en una relación de generalización. Todos los hijos representan una cualidad dada por la que especializar al padre, en contraste a otras cualidades potenciales por las que especializar al mismo padre; representa una dimensión de la especialización.

Véase también generalización, supratipo, pseudoatributo.

Semántica

A veces, un elemento del modelo se puede especializar según diversas cualidades. Cada cualidad representa una dimensión ortogonal independiente de la especialización. Por ejemplo, un vehículo se puede especializar por la propulsión (motor de gasolina, motor espacial, viento, animal, humano), así como por el lugar donde se mueve (tierra, agua, aire, espacio exterior). Un discriminador es el nombre de una dimensión de la especialización. Un elemento se puede especializar en múltiples dimensiones, que deben estar presentes en una instancia concreta.

Una relación de generalización puede tener un discriminador: una cadena que represente una dimensión para clasificar a los hijos del padre. Todas las relaciones de especialización de un solo padre con el mismo nombre de discriminador forman un grupo; cada grupo es una dimensión separada de la especialización. El conjunto completo de nombres del discriminador representa el conjunto completo de dimensiones para especializar al padre. Una instancia debe ser simultáneamente una instancia de un hijo de cada grupo del discriminador. Por ejemplo, un vehículo debe tener una propulsión y un lugar.

Cada discriminador representa una cualidad abstracta del padre, una cualidad que es especializada por los hijos que llevan esa relación del discriminador al padre. Pero un padre con discriminadores múltiples tiene dimensiones múltiples, que se deben especializar para producir un elemento concreto. Por lo tanto, los hijos dentro de un grupo del discriminador son intrínsecamente abstractos. Cada uno de ellos es solamente una descripción parcial del padre, una descripción que acentúe una cualidad e ignore el resto.

Por ejemplo, una subclase de vehículo que se centra en la propulsión omite el lugar. Un elemento concreto requiere la especialización de todas las dimensiones simultáneamente. Esto puede ocurrir por la herencia múltiple del elemento concreto del modelo de un hijo en cada una de las dimensiones, o por la clasificación múltiple de una instancia de un hijo en cada una de las dimensiones. Hasta que se combinan todos los discriminadores, la descripción sigue siendo abstracta.

Por ejemplo, un vehículo real debe tener un medio de propulsión y un lugar. Un vehículo con propulsión por viento y que se mueve por el agua es un barco de vela. No hay nombre particular para un vehículo de tracción animal que se mueva por el aire, pero las instancias de la combinación existen en la fantasía y en la mitología.

La ausencia de una etiqueta de discriminador indica el discriminador «empty» (vacío), que también se considera un discriminador válido (el discriminador por defecto). Esta convención permite que el caso usual no discriminado sea tratado uniformemente. En efecto, si ninguna de las líneas de generalización tiene discriminadores, entonces todos los hijos están en el mismo discriminador. Es decir, hay un discriminador al cual pertenecen todas las especializaciones, y produce la misma semántica que el caso sin discriminador.

Estructura

Cada arco de especialización (generalización) tiene una cadena de discriminador, que puede ser la cadena vacía.

El discriminador es un pseudoatributo del padre. Debe ser único entre los atributos y los roles de la asociación del padre. El dominio para el pseudoatributo es el conjunto de clases hijas. Las múltiples incidencias del mismo nombre de discriminador se permiten entre hijos diferentes y el padre e indican que los hijos pertenecen a la misma partición.

Notación

Un discriminador se muestra como una etiqueta de texto en una flecha de generalización (Figura 13.72). Si dos arcos de generalización con el mismo discriminador comparten una punta de flecha, el discriminador se puede colocar en la punta de flecha.

Ejemplo

La Figura 13.72 muestra una especialización de **Empleado** en dos dimensiones: **estado** del empleado y **localidad**. Cada dimensión tiene el rango de los valores representados por las subclases. Pero son necesarias ambas dimensiones para producir una subclase instanciable. El **Enlace**, por ejemplo, es una clase que es ambos, **Supervisor** y **Expatriado**.

Cualquier descendiente de una sola dimensión es abstracto, hasta que las dos dimensiones se recombinan en un descendiente con herencia múltiple.

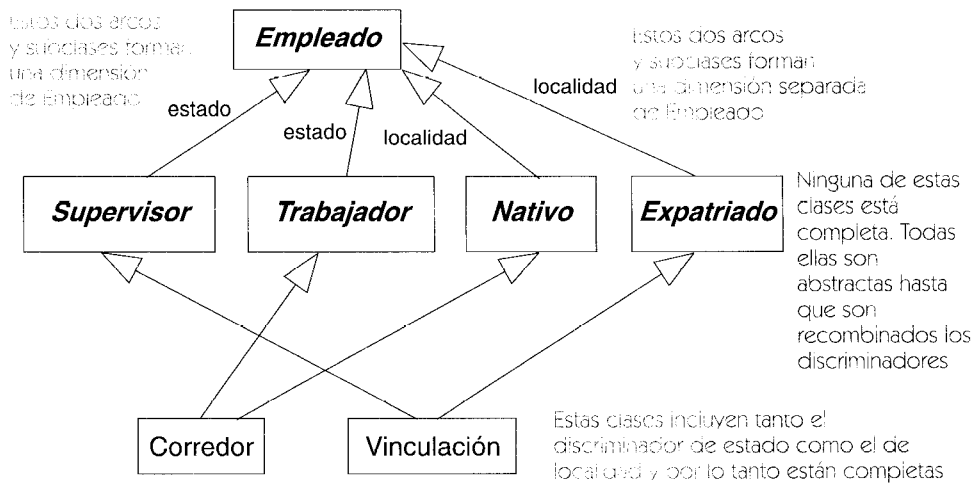


Figura 13.72 Discriminadores

diseño

Esa etapa de un sistema que describe cómo se implementará el sistema, en un nivel lógico sobre código real. En el diseño, las decisiones estratégicas y tácticas se toman para resolver los requisitos funcionales y de calidad requeridos de un sistema. Los resultados de esta etapa son representados por los modelos a nivel de diseño, especialmente la vista estática, vista de la máquina de estados, y vista de interacción. Contrastar con: análisis, diseño, implementación, y despliegue.

Véase fases de modelado, proceso de desarrollo.

disparador

Evento cuya aparición hace que una transición pueda dispararse. Se puede emplear la aplicación como nombre (para denotar el efecto en sí) o como verbo (para denotar la aparición del evento³).

Véase también transición de finalización, transición.

Semántica

Toda transición (salvo una transición de finalización, que se dispara al finalizar alguna actividad interna) hace referencia a un evento que forma parte de su estructura. Si se produce el evento

³ En inglés, la palabra "trigger" significa tanto "disparador" como "disparar". *N. del T.*

cuando el objeto se encuentra en un estado que contenga una transición saliente cuyo disparador sea ese evento o un antecesor del evento, entonces se comprueba la condición de guarda de la transición. Si se cumple la condición, entonces se permite que se dispare la transición. Si hay más de una transición disponible para ser disparada, sólo llega a dispararse una. La selección puede no ser determinista. (Si el objeto posee más de un estado concurrente, se puede disparar una transición desde cada estado, pero como máximo se puede disparar una transición desde cada estado.)

Observe que la condición de guarda sólo se comprueba una vez, en el momento en que se produce el evento disparador (incluyendo los eventos implícitos de finalización). Si no se habilita para el disparo ninguna transición al producirse un evento, entonces se ignora el evento. Esto no es un error.

Los parámetros del evento disparador se pueden utilizar en la condición de guarda o en una acción asociada a la transición, así como en una acción de entrada en el estado destino.

A lo largo de la ejecución de un paso que se ejecuta hasta finalizar realizado después de una transición, el evento disparador sigue estando disponible como evento actual para las acciones de los subpasos de la transición. El tipo exacto de este evento puede ser desconocido en una acción de entrada o en algún segmento posterior de una transición de múltiples segmentos. Por tanto, el tipo de evento se puede discriminar en la acción empleando una operación polimorfa o una sentencia de selección (como *case* o *switch*). Una vez conocido el tipo exacto del evento, se pueden utilizar sus parámetros.

Notación

El nombre y signatura del evento disparador forman parte de la etiqueta de la transición.

Véase transición.

Se puede acceder al evento disparador desde una expresión mediante la palabra clave **currentEvent**. Esta palabra clave se refiere al evento disparador del primer segmento en las transiciones de múltiples segmentos.

disparar

Ejecutar una transición.

Véase también atómico/a, disparador.

Semántica

Cuando ocurre un evento requerido por una transición, y la condición de guarda en la transición está satisfecha, la transición realiza su acción y cambia el estado activo.

Cuando un objeto recibe un evento, se guarda el evento si la máquina de estados está ejecutando una etapa atómica. Cuando se termina la etapa, la máquina de estados gestiona el evento que ha ocurrido. Una transición se activa si su evento se maneja mientras el propio

objeto está en el estado que contiene la transición o en un subestado anidado dentro del estado que contiene la transición. Un evento satisface un evento de disparador cuyo tipo sea un antecesor del tipo del evento que ocurre. Si una transición compleja tiene múltiples estados de origen, todos deben estar activos para que la transición esté disponible. Una transición de terminación se activa cuando su estado origen termina su actividad. Si es un estado compuesto, está disponible cuando todos sus subestados directos han terminado o han alcanzado sus estados finales.

Cuando se trata el evento, se evalúa la condición de guarda (si existe). Si la expresión booleana en la condición de guarda se evalúa como verdadera, entonces la transición se dispara. La acción en la transición se ejecuta, y el estado del objeto se convierte en el estado destino de la transición (sin embargo, no ocurre ningún cambio de estado en una transición interna). Durante el cambio de estado, cualquiera que sean acciones de salida y acciones de entrada en la ruta mínima, desde el estado original del objeto al estado destino, se ejecutan las transiciones. Observe que el estado original puede ser un subestado anidado del estado origen de la transición.

Si la condición de guarda no está satisfecha, nada sucede como resultado de esta transición, aunque alguna otra transición podría dispararse si sus condiciones están satisfechas.

Aunque más de una transición sea elegible para ser disparada, sólo una de ellas se disparará. Una transición en un estado anidado tiene prioridad sobre una transición en un estado que incluye. Si no, la elección de transición es indefinida y puede ser no determinista. Esto es a menudo una situación del mundo real.

Como cuestión práctica, una implementación puede proporcionar una ordenación de las transiciones para dispararlas. Esto no cambia la semántica, pues el mismo efecto podría ser alcanzado organizando las condiciones de guarda de modo que no se solapen. Pero es a menudo más simple poder decir "Esta transición solamente se dispara si ninguna otra transición se dispara".

Eventos diferidos. Si el evento o uno de sus antecesores está marcado como diferido en el estado o en un estado lo que incluye, y el evento no acciona una transición, el evento es un evento diferido hasta que el objeto pasa a un estado en el cual el evento no se difiera. Cuando el objeto pasa a un nuevo estado, cualquier evento previamente diferido pendiente que ya no sea diferido, pasa a estar pendiente y ocurren en un orden indeterminado. Si el primer evento pendiente no causa que se dispare una transición, se ignora y otro ocurre evento pendiente. Si un evento previamente diferido está marcado para el aplazamiento en el nuevo estado, puede accionar una transición, pero sigue diferido si no puede accionar una transición. Si la ocurrencia de un evento causa una transición a un nuevo estado, cualquier evento pendiente restante o evento diferido son evaluados de acuerdo con el estado de aplazamientos del nuevo estado y se establece un nuevo conjunto de eventos pendientes.

Una implementación puede imponer reglas más terminantes sobre el orden en el que los eventos diferidos se procesan u ofrecer operaciones para manipular dicho orden.

división

Una transición compleja en la cual un estado origen es sustituido por dos o más estados destino, dando por resultado un aumento en el número de estados activos. Antónimo: unión.

Véase también transición compleja, estado compuesto, unión.

Semántica

Una división es una transición con un estado origen y dos o más estados de destino. Si el estado origen está activo y ocurre el evento disparador, se ejecuta la acción de la transición y todos los estados destino pasan a estar activos. Los estados destino deben estar en diversas regiones de un estado compuesto concurrente.

Notación

Una división se representa como una línea gruesa a la que llega una flecha de la transición y salen dos o más flechas de transición. Puede tener una etiqueta de transición (condición de guarda, evento disparador, y acción). La Figura 13.73 muestra una división explícita a un estado compuesto concurrente.

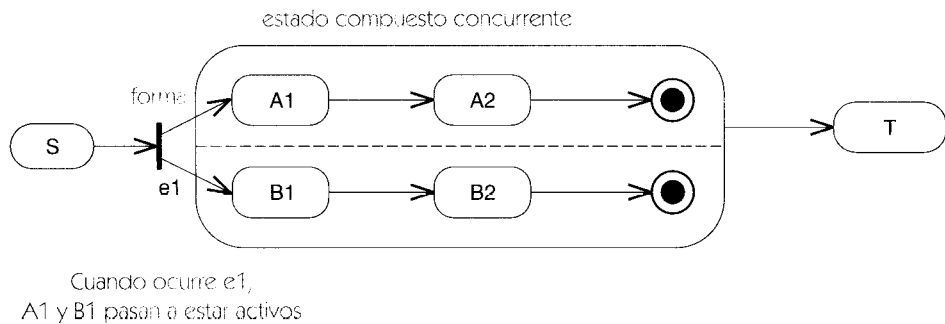


Figura 13.73 División

eficiencia de navegación

Término que indica si es o no posible recorrer eficientemente una asociación binaria partiendo de un objeto para obtener el objeto o conjunto de objetos asociados con él. El concepto no es aplicable a las asociaciones n -arias. La eficiencia de navegación está relacionada con la navegabilidad, pero no es una propiedad definitoria.

Véase también navegabilidad.

Semántica

Se puede definir la eficiencia de navegación en forma general, para que sea aplicable al diseño abstracto y también a distintos lenguajes de programación. Diremos que una asociación binaria es navegable eficientemente si el coste medio necesario para obtener el conjunto de objetos asociados es proporcional al número de objetos del conjunto (no al límite superior de la multiplicidad, que podría ser infinito) más una constante determinada. En términos de complejidad computacional, el coste es $O(n)$. Si la multiplicidad es uno o cero-uno, entonces el coste de acceso tiene que ser constante, lo cual impide efectuar búsquedas en una lista de longitud variable. Una definición ligeramente más imprecisa de la eficiencia de navegación permitiría un coste mínimo de $\log(n)$.

Aun cuando las asociaciones navegables de multiplicidad uno suelen implementarse empleando un puntero incrustado en el bloque que contiene los atributos del objeto, es posible una implementación externa empleando tablas hash, que tienen un coste medio de acceso constante. Por tanto, se puede implementar una asociación como un objeto de tabla de búsqueda externo con respecto a las clases participantes y se puede seguir considerando navegable. (En algunas situaciones de tiempo real, es preciso limitar el tiempo en el caso peor y no el tiempo medio. Esto no requiere un cambio en la definición básica salvo cambiar el tiempo medio por el tiempo en el caso peor, pero pueden ser descartados los algoritmos probabilísticos como las tablas hash.)

Si una asociación no es navegable en una dirección dada, esto no significa que no se pueda recorrer en absoluto, sino que el coste del recorrido puede ser significativo —por ejemplo, quizá requiera efectuar búsquedas en una larga lista—. Si el acceso en una dirección es infrecuente, una búsqueda puede ser una opción razonable. La eficiencia de navegación es un concepto de diseño que permite al diseñador diseñar las rutas de acceso a objetos teniendo una comprensión de los costes de complejidad computacional. Normalmente, la navegabilidad implica la eficiencia de navegación.

Resulta posible (aun siendo relativamente raro) tener una asociación que no sea navegable eficientemente en ninguna dirección. Esta asociación podría estar implementada como una lista de enlaces que es preciso estudiar para realizar un recorrido en cualquier dirección. Sería posible recorrerla, pero sería poco eficiente. Sin embargo, la utilidad de una de estas asociaciones es reducida.

Discusión

La eficiencia de navegación indica la eficiencia que se tiene al obtener el conjunto de objetos que están relacionados con un objeto dado. Cuando la multiplicidad es 0..1 ó 1, entonces la implementación evidente es un puntero en el objeto origen. Cuando la multiplicidad es muchos, entonces la implementación normal es una clase contenedora que alberga un conjunto de punteros. La clase contenedora, a su vez, puede residir o no en el registro de datos de un objeto de la clase, dependiendo de si se puede o no obtener con coste constante (que es lo que suele suceder para el acceso a través de punteros). La clase contenedora debe tener la propiedad de eficiencia de navegación. Por ejemplo, una lista simple de todos los enlaces de una asociación no sería eficiente, porque los enlaces de un objeto estarían mezclados con muchos otros enlaces carentes de interés y sería preciso hacer una búsqueda. Una lista de enlaces almacenada en cada objeto sí sería eficiente, porque no se requeriría una búsqueda innecesaria.

En una asociación cualificada, una indicación de navegabilidad en la dirección que sale del calificador suele indicar que es eficiente obtener el objeto o conjunto de objetos seleccionado por un objeto origen y un valor del calificador. Esto es consistente con una implementación que haga uso de tablas hash o quizá con un árbol binario indexado por el valor del calificador (que es precisamente la razón por la que se incluyen los calificadores como concepto de modelado).

ejecutar hasta finalizar

Transición o serie de acciones que tiene que acabar por completo.

Véase también acción, atómico/a, máquina de estados, transición.

Semántica

En una máquina de estados, ciertas acciones o series de acciones son atómicas: no se pueden hacer acabar, ni abortar, ni pueden ser interrumpidas por otras acciones. Cuando se dispara una transición, todas las acciones asociadas a ella o que hayan sido invocadas por ella deben terminarse como un grupo, incluyendo las acciones de entrada y las acciones de salida de aquellos estados en los que entre o de los que salga. Se dice que la ejecución de una transición se ejecuta hasta finalizar porque no espera para admitir otros eventos.

La semántica de ejecutar-hasta-finalizar puede contrastarse con la semántica de los estados normales. Cuando está activo un estado, un evento puede producir una transición a otro estado. Toda actividad de ese estado será abortada por la transición.

Una transición puede estar compuesta por múltiples segmentos organizados en forma de cadena y separados mediante pseudoestados. Un grupo de varias cadenas se puede separar o fusionar, así que el modelo global puede contener un grafo de segmentos separados por pseudoestados. Sólo el primer segmento de la cadena puede tener un evento disparador. La transición se dispara cuando la máquina de estados maneja el evento disparado. Si se satisfacen las condiciones de guarda de todos los segmentos, entonces la transición está habilitada y se dispara, siempre y cuando no se dispare otra transición. Se ejecutan las acciones de los sucesivos segmentos. Una vez comenzada la ejecución, es preciso ejecutar las acciones de todos los segmentos antes de que haya finalizado el paso de ejecución hasta finalizar.

Durante la ejecución de un paso que se ejecuta hasta finalizar, el evento disparador que haya iniciado la transición está disponible para las acciones como evento en curso. Las acciones de entrada y salida pueden por tanto obtener argumentos del evento disparador. Los distintos eventos pueden dar lugar a una acción de entrada o de salida, pero una acción puede discriminar el tipo de suceso en curso en una sentencia de caso.

Como consecuencia de la semántica que poseen las acciones en el caso de una ejecución hasta finalización, debería utilizarse para modelar asignaciones, indicadores, aritmética sencilla y otros tipos de operaciones de mantenimiento doméstico. Los cálculos largos deberían modelarse como actividades interrumpibles.

elaboración

La segunda fase de un proceso de desarrollo de software, durante el cual el diseño del sistema ha empezado y la arquitectura es desarrollada y probada. Durante esta fase, se completa la mayor parte de la vista del análisis, junto con las partes arquitectónicas de la vista del diseño. Si se construye un prototipo ejecutable, se hace algo de la vista de la implementación.

Véase proceso de desarrollo.

elemento

Un componente atómico de un modelo. Este libro describe los elementos que pueden utilizarse en los modelos de UML —elementos del modelo, que expresan información semántica, y